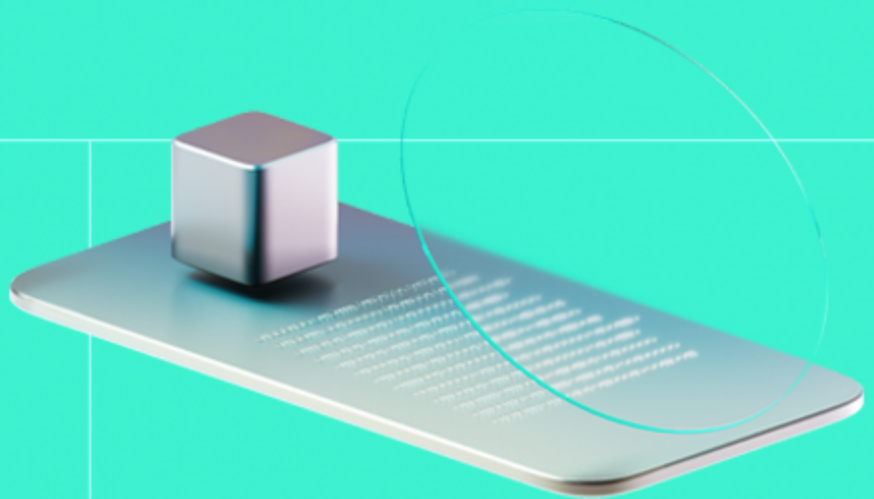




Smart Contract Code Review And Security Analysis Report

Customer: BullBit

Date: 22/01/2026



We express our gratitude to the BullBit team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

A hybrid trading protocol with off-chain execution and on-chain settlement: the off-chain sequencer handles order matching and trading operations, while on-chain smart contracts serve as the secure treasury, managing assets and settling deposits/withdrawals through batched updates from the **Verifier** contract.

Document

Name	Smart Contract Code Review and Security Analysis Report for BullBit
Audited By	Seher Saylik, Kornel Światłowski
Approved By	Ataberk Yavuzer
Website	https://alpha.bullbit.ai/
Changelog	06/01/2026 - Preliminary Report
	22/01/2026 - Final Report
Platform	Base
Language	Solidity
Tags	Treasury, Signatures, Upgradable
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/bullbit/Protocol-V1
Initial Commit	7766cc0501b540660fa8845f23592eb7620266f4
Final Commit	322258e4622b2430f1249853f2c0c013a038548f

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

26

Total Findings

19

Resolved

5

Accepted

2

Mitigated

Findings by Severity

Severity

Count

Critical

0

High

2

Medium

9

Low

5

Vulnerability	Severity	Status
F-2025-14436 - Blacklisted Token Recipient Permanently Blocks FIFO Forced Withdrawals	High	Fixed
F-2026-14500 - finalizeForceWithdrawal Silently Burns User Balance When Pool Has Insufficient Token Balance	High	Fixed
F-2025-14443 - Uninitialized minWithdrawAmount Allows Zero-Amount Forced Withdrawal Requests To Block Queue Processing	Medium	Fixed
F-2025-14448 - Uncoordinated Escape Hatch Mechanisms Cause Permanent forcedWithdrawalRequests Lock When InclusionQueue Executes First	Medium	Fixed
F-2025-14461 - Fee-on-Transfer / Rebasing Tokens Break Accounting	Medium	Fixed
F-2025-14485 - Lack of Limits and Delay in Forced Withdrawal Parameter Updates	Medium	Fixed
F-2025-14487 - Forced Withdrawal Flow Is Not Fully Censorship-Resistant	Medium	Fixed
F-2025-14494 - InclusionQueue Forced Withdrawal Mechanism Limited to USDC	Medium	Accepted
F-2026-14499 - Cross-Chain Deposit Credits Not Backed by Locked Assets	Medium	Accepted
F-2026-14505 - Inconsistent Asset Accounting Between Internal Balances and Contract Holdings	Medium	Accepted
F-2026-14523 - Pending Force Withdrawal Requests Removed On Balance Update	Medium	Fixed
F-2025-14396 - Inconsistent System Configuration Due to Non-Atomic Governance Updates	Low	Fixed

Vulnerability	Severity	Status
F-2025-14397 - Non-Refundable Fee Loss and Queue Spam in InclusionQueue Contract	Low	Accepted
F-2025-14452 - No Signature Expiry/Deadline	Low	Fixed
F-2025-14462 - notContract Modifier Is Bypassable and Breaks Modern Wallet Support	Low	Fixed
F-2025-14469 - Use of transfer() Instead of call() to Send Native Assets	Low	Fixed
F-2025-14463 - Missing Basic Validation for Forced-Withdrawal Destination	Info	Fixed
F-2025-14468 - Sequential Queue Processing Forces Users to Pay for Others' Withdrawals	Info	Fixed
F-2025-14477 - Missing Event Emissions for Important State Changes	Info	Fixed
F-2025-14481 - Redundant forcedQueueUids Array Parameter	Info	Fixed
F-2026-14512 - Floating Pragma	Info	Fixed
F-2026-14513 - Custom Errors Can Be Used for Gas Efficiency	Info	Mitigated
F-2026-14514 - Lack of Two-Step Ownership Transfer	Info	Fixed
F-2026-14518 - Violation of the Checks-Effects-Interactions (CEI) pattern	Info	Mitigated
F-2026-14519 - Unused Pausable Inheritance in WithdrawalManager	Info	Fixed
F-2026-14520 - Shared Pool Nonce Invalidates Pending Signatures	Info	Accepted

Documentation quality

- Documentation was provided; however, parts of it reference outdated features.
- Technical description is not provided.

Code quality

- The development environment was configured.

Test coverage

Code coverage of the project is **93.23%** (branch coverage).

- Basic user interactions were tested.
- Edge cases including race conditions, out-of-order processing were covered.
- Privileged functions were tested thoroughly.

Table of Contents

System Overview	7
Privileged Roles	8
Potential Risks	11
Findings	13
Vulnerability Details	13
Disclaimers	80
Appendix 1. Definitions	81
Severities	81
Potential Risks	81
Appendix 2. Scope	82
Appendix 3. Additional Valuables	83

System Overview

The **Governance** contract is the central administration contract that manages and configures all system module addresses and the Sequencer address. It stores references to all core contracts and provides a link `setContracts()` to link all contracts together, along with granular configuration functions for individual modules. It manages forced withdrawal parameters and cross-chain asset settings. The contract is upgradable using the UUPS pattern, with upgrade authorization restricted to the owner.

The **Verifier** contract is the entry point for the Sequencer to submit batched state updates from the off-chain execution layer to the on-chain settlement layer. It processes batched **Pool** updates approximately every 5 seconds, including cross-chain deposits, P/L updates, withdrawal requests with signature verification, and forced withdrawals from the **InclusionQueue**. It also processes batched **Vault** updates with USDC balance changes, withdrawal requests, and forced withdrawals. The contract maintains batch nonces for sequential processing and tracks last update timestamps to detect sequencer inactivity. It provides an escape hatch for processing expired queue items and enforces limits on forced withdrawals per batch. The contract is upgradable using the UUPS pattern, with upgrade authorization restricted to the owner.

The **Pool** contract manages user on-chain spot and margin trading balances for multiple tokens. Users can deposit tokens directly on-chain. The **Verifier** updates profit and loss through state changes and credits cross-chain deposits. **WithdrawalManager** and **Verifier** execute withdrawals. Users can initiate forced withdrawals when the sequencer is inactive. The contract is upgradable using the UUPS pattern, with upgrade authorization restricted to the owner.

The **Vault** contract manages user on-chain spot and margin trading balances for USDC. Users can deposit USDC directly, and the **Verifier** updates profit and loss through state changes. **WithdrawalManager** and **Verifier** can execute USDC withdrawals. Users can initiate forced withdrawals when the sequencer is inactive. The contract is upgradable using the UUPS pattern, with upgrade authorization restricted to the owner.

The **WithdrawalManager** contract handles EIP-712 signature-based authentication for withdrawal requests and dispatches them to **Pool** or **Vault** contracts. It verifies signatures for **Pool** on-chain withdrawals, **Pool** cross-chain withdrawals, and **Vault** withdrawals. The contract maintains nonces per user to prevent replay attacks and tracks cross-chain assets. It processes withdrawal requests on behalf of the **Verifier** during batch updates. The contract is upgradable using the UUPS pattern, with upgrade authorization restricted to the owner.

The **InclusionQueue** contract provides a decentralized forced withdrawal queue mechanism that allows users to bypass centralized control when the sequencer is inactive. It maintains separate queues for **Pool** and **Vault** withdrawals. Users can submit forced withdrawal requests, and the contract charges a fee in USDC and enforces minimum withdrawal amounts. The contract is upgradable using the UUPS pattern, with upgrade authorization restricted to the owner.

The **Pausable** contract provides pause and unpause functionality for emergency situations and is used as a base contract by all main system contracts. It allows the owner to toggle the pause state and provides modifiers for access control. The contract is inherited by **Pool**, **Vault**, **Verifier**, **InclusionQueue**, and **WithdrawalManager**.

Privileged roles

Each contract contains a combination of access control mechanisms — `OwnableUpgradeable` for `onlyOwner` access, and a custom implementation of access restricted to contract addresses stored in the contract's storage variables. Each contract has the following privileged roles:

Governance

- Owner
 - `_authorizeUpgrade()` - Authorizes contract upgrades via the UUPS pattern
 - `setContracts()` - Configures all system contract addresses and links them together
 - `setQueueInfo()` - Updates InclusionQueue contract addresses (governance, verifier, pool, vault)
 - `setPoolInfo()` - Updates Pool contract addresses (governance, verifier, withdrawal manager)
 - `setVaultInfo()` - Updates Vault contract addresses (governance, verifier, withdrawal manager)
 - `setVerifierInfo()` - Updates Verifier contract addresses (governance, sequencer, pool, vault, withdrawal manager, queue)
 - `setVerifierForcedInfo()` - Sets forced withdrawal parameters for Verifier (max forced per batch, forced deadline)
 - `setWMInfo()` - Updates WithdrawalManager contract addresses (governance, verifier, pool, vault)
 - `setWMAssetCrossChain()` - Configures which tokens are cross-chain assets in WithdrawalManager
 - `setPoolForcedInfo()` - Sets forced withdrawal parameters for Pool (sequencer inactivity threshold, force withdraw delay)
 - `setVaultForcedInfo()` - Sets forced withdrawal parameters for Vault (sequencer inactivity threshold, force withdraw delay)

Pool

- Owner
 - `_authorizeUpgrade()` - Authorizes contract upgrades via the UUPS pattern
 - `togglePause()` - Pauses or unpauses contract operations
- Governance
 - `setInfo()` - Updates contract addresses (governance, verifier, withdrawal manager)
 - `setForcedInfo()` - Sets forced withdrawal parameters (sequencer inactivity threshold, force withdraw delay)
- Verifier
 - `applyStateChanges()` - Updates user P/L balances by applying positive or negative deltas per token
 - `creditCrossChainDeposit()` - Credits user balance for cross-chain deposits
 - `executeOnChainWithdrawal()` - Executes on-chain withdrawals by transferring tokens to users
- WithdrawalManager
 - `executeOnChainWithdrawal()` - Executes on-chain withdrawals by transferring tokens to users
 - `executeCrossChainWithdrawal()` - Executes cross-chain withdrawals by subtracting balance and emitting an event

Vault

- Owner

- `_authorizeUpgrade()` - Authorizes contract upgrades via the UUPS pattern
- `togglePause()` - Pauses or unpauses contract operations
- Governance
 - `setInfo()` - Updates contract addresses (governance, verifier, withdrawal manager)
 - `setForcedInfo()` - Sets forced withdrawal parameters (sequencer inactivity threshold, force withdraw delay)
- Verifier
 - `applyStateChanges()` - Updates LP P/L balances by applying positive or negative deltas
 - `executeOnChainWithdrawal()` - Executes LP withdrawals by transferring USDC to users
- WithdrawalManager
 - `executeOnChainWithdrawal()` - Executes LP withdrawals by transferring USDC to users

Verifier

- Owner
 - `_authorizeUpgrade()` - Authorizes contract upgrades via the UUPS pattern
 - `togglePause()` - Pauses or unpauses contract operations
- Governance
 - `setInfo()` - Updates contract addresses (governance, sequencer, pool, vault, withdrawal manager, queue)
 - `setForcedInfo()` - Sets forced withdrawal parameters (max forced per batch, forced deadline)
- Sequencer
 - `submitPoolUpdateBatch()` - Submits batched Pool updates, including cross-chain deposits, P/L updates, withdrawal requests, and forced withdrawals
 - `submitVaultUpdateBatch()` - Submits batched Vault updates, including LP P/L updates, withdrawal requests, and forced withdrawal

WithdrawalManager

- Owner
 - `_authorizeUpgrade()` - Authorizes contract upgrades via the UUPS pattern
 - `togglePause()` - Pauses or unpauses contract operations
- Governance
 - `setInfo()` - Updates contract addresses (governance, verifier, pool, vault)
 - `setAssetCrossChain()` - Configures which tokens are cross-chain assets
- Verifier
 - `processOnChainPoolWithdrawal()` - Verifies EIP-712 signature and processes on-chain Pool withdrawals
 - `processCrossChainPoolWithdrawal()` - Verifies EIP-712 signature and processes cross-chain Pool withdrawals
 - `processVaultWithdrawal()` - Verifies EIP-712 signature and processes Vault withdrawals

InclusionQueue

- Owner
 - `_authorizeUpgrade()` - Authorizes contract upgrades via the UUPS pattern
 - `togglePause()` - Pauses or unpauses contract operations
 - `withdrawFee()` - Withdraws collected USDC fees from the contract

- `setAmount()` - Updates the minimum withdrawal amount and fee amount
- `withdrawToken()` - Withdraws any ERC20 token from the contract
- `withdrawMain()` - Withdraws native tokens from the contract
- Governance
 - `setInfo()` - Updates contract addresses (governance, verifier, pool, vault)
- Verifier
 - `setPoolQueueIndex()` - Updates the Pool queue processing index after processing requests
 - `setVaultQueueIndex()` - Updates the Vault queue processing index after processing requests

Potential Risks

- **Off-Chain Trust Assumption in Cross-Chain Withdrawals:** When `processCrossChainPoolWithdrawal()` is executed, the user's on-chain balance in `Pool.balances[user][token]` is immediately decremented, but no actual token transfer occurs — the function only emits a `CrossChainWithdrawal` event containing the destination address. Users must fully trust that an off-chain bridge system will observe this event and deliver the equivalent tokens to the specified destination chain. If the off-chain bridge system fails, is compromised, experiences downtime, or never processes the withdrawal event, the user's balance is permanently lost with no on-chain recourse or refund mechanism.
- **Unrestricted Sequencer Authority Over User Balances:** The sequencer has unilateral authority to modify any user's balance via `submitPoolUpdateBatch()` through arbitrary P/L deltas (`applyStateChanges()`) and unverified cross-chain deposit credits (`creditCrossChainDeposit()`), with no on-chain proof or validation mechanism. A compromised or malicious sequencer can drain user funds by crediting fake deposits to an attacker address and withdrawing real tokens, or by applying negative deltas to victim accounts. Users must fully trust the sequencer's integrity, as the escape hatch mechanisms can be circumvented by a minimally active sequencer applying small P/L updates to cancel pending forced withdrawal requests.
- **Dynamic Array Iteration Gas Limit Risks:** The project iterates over large dynamic arrays, which leads to excessive gas costs, risking denial of service due to out-of-gas errors, directly impacting contract usability and reliability.
- **External Calls in Iteration Risks:** Making external calls within loops increases the risk of gas exhaustion, potentially leading to failed transactions and reduced contract reliability, especially when processing large datasets.
- **Owner's Unrestricted State Modification:** The absence of restrictions on state variable modifications by the owner leads to arbitrary changes, affecting contract integrity and user trust, especially during critical operations like minting phases.
- **Absence of Time-lock Mechanisms for Critical Operations:** Without time-locks on critical operations, there is no buffer to review or revert potentially harmful actions, increasing the risk of rapid exploitation and irreversible changes.
- **Single Points of Failure and Control:** The project is fully or partially centralized, introducing single points of failure and control. This centralization can lead to vulnerabilities in decision-making and operational processes, making the system more susceptible to targeted attacks or manipulation.
- **Administrative Key Control Risks:** The digital contract architecture relies on administrative keys for critical operations. Centralized control over these keys presents a significant security risk, as compromise or misuse can lead to unauthorized actions or loss of funds.
- **Single Entity Upgrade Authority:** The token ecosystem grants a single entity the authority to implement upgrades or changes. This centralization of power risks unilateral decisions that may not align with the community or stakeholders' interests, undermining trust and security.
- **Flexibility and Risk in Contract Upgrades:** The project's contracts are upgradable, allowing the administrator to update the contract logic at any time. While this provides flexibility in addressing issues and evolving the project, it also introduces risks if upgrade processes are not properly managed or secured, potentially allowing for unauthorized changes that could compromise the project's integrity and security.
- **Absence of Upgrade Window Constraints:** The contract suite allows for immediate upgrades without a mandatory review or waiting period, increasing the risk of rapid deployment of malicious

or flawed code, potentially compromising the system's integrity and user assets.

- **Non-Refundable Queue Fee:** Forced withdrawal requests are always consumed from the queue regardless of execution outcome, meaning users lose their queue fee even if the withdrawal fails (due to contract insolvency, transfer errors, or blacklisting) or executes with a partial amount less than requested. The client stated that this is an intentional design trade-off to prevent head-of-line blocking and ensure queue liveness for all users.

Findings

Vulnerability Details

[F-2025-14436](#) - Blacklisted Token Recipient Permanently Blocks FIFO Forced Withdrawals - High

Description:

The protocol implements a forced withdrawal queue (`InclusionQueue`) as an escape hatch mechanism, allowing users to bypass sequencer control when needed. The `Verifier` contract processes these requests in strict FIFO order, requiring:

1. Processing must start from the next unprocessed index (`nextPoolRequestToProcessIndex` / `nextVaultRequestToProcessIndex`)
2. Requests must be processed contiguously (no skipping)

The vulnerability arises because:

1. Token transfers use `safeTransfer` which reverts on failure
2. Tokens like USDC/USDT implement address blacklisting (for OFAC sanctions, fraud, etc.)
3. If a blacklisted address is at the front of the queue, the transfer reverts
4. Due to FIFO constraints, no subsequent requests can be processed
5. There is no mechanism to skip or mark failed requests

`Pool.sol` - `executeOnChainWithdrawal()`

```
function executeOnChainWithdrawal(
    address user,
    address token,
    uint256 amount
) external onlyWM0rVRF {
    require(user != address(0), "Pool: user 0x");
    require(token != address(0), "Pool: token 0x");
    require(amount > 0, "Pool: amount = 0");
    uint256 bal = balances[user][token];
    require(bal >= amount, "Pool: insufficient");
    balances[user][token] = bal - amount;
    IERC20(token).safeTransfer(user, amount); // REVERTS if user is black
listed
    emit OnChainWithdrawal(user, token, amount, balances[user][token]);
}
```

`Vault.sol` - `executeOnChainWithdrawal()`

```

function executeOnChainWithdrawal(
    address user,
    uint256 amount
) external onlyWM0rVRF {
    require(user != address(0), "Vault: user 0x");
    require(amount > 0, "Vault: amount = 0");
    uint256 bal = balances[user];
    require(bal >= amount, "Vault: insufficient");
    balances[user] = bal - amount;
    IERC20(usdcToken).safeTransfer(user, amount); // REVERTS if user is b
lacklisted
    emit Withdrawal(user, amount, balances[user]);
}

```

Both `submitPoolUpdateBatch()` and `submitVaultUpdateBatch()` enforce strict sequential ordering of forced withdrawals: the first queue ID must match the current pointer (`nextPoolRequestToProcessIndex` / `nextVaultRequestToProcessIndex`), and all subsequent IDs must be contiguous. When iterating through requests, each calls `executeOnChainWithdrawal()` on the respective contract. If any transfer reverts—such as when the recipient is blacklisted—the entire batch fails. The queue pointer only advances after all transfers succeed, meaning a single blacklisted user permanently blocks everyone behind them.

```

function submitPoolUpdateBatch(...){

    // 3) forced withdraw from queue (POOL)
    uint256 lenForced = forcedQueueIds.length;
    if (lenForced > 0) {
        require(lenForced <= maxForcedPerBatch, "VRF: too many forced");
        uint256 nextIdx = IQueueForVerifier(inclusionQueueContract).nextPoolRequestToProcessIndex();
        // Ensure forced ids start from nextIdx and continuous
        require(forcedQueueIds[0] == nextIdx, "VRF: forced must start at pointer");
        for (uint256 k = 0; k < lenForced; k++) {
            require(forcedQueueIds[k] == nextIdx + k, "VRF: forced not contiguous");
            (, address u, address token, uint256 amount, uint256 ts) = IQueueForVerifier(inclusionQueueContract).poolQueue(forcedQueueIds[k]);
            require(block.timestamp >= ts + forcedDeadline, "VRF: not overdue");
            uint256 bal = IPoolForVerifier(poolContract).balances(u, token);
            if (bal >= amount) {
                IPoolForVerifier(poolContract).executeOnChainWithdrawal(u, token, amount); // @audit Reverts here if blacklisted
            }
        }
    }
}

```

```

emit OverdueRequestProcessed(forcedQueueIds[k], u, 0, amou
nt);

    } else {
        emit OverdueRequestProcessed(forcedQueueIds[k], u, 0, 0);
    }
}

IQueueForVerifier(inclusionQueueContract).setPoolQueueIndex(lenFor
ced);
}

```

The `processOverdueQueueItem()` function, designed as a public escape hatch for when the sequencer is unresponsive, suffers from the same flaw. It always processes from the current queue pointer and only advances it after successful execution. Users cannot specify which request to process—they must process whatever is next in line. If that request belongs to a blacklisted address, the function reverts before updating the index. Ironically, the escape hatch mechanism itself becomes permanently blocked.

Assets:

- ./Pool.sol [zip file]
- ./Vault.sol [zip file]

Status:

Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: High

Recommendations

Remediation: Wrap transfer calls in try-catch blocks and advance the queue pointer regardless of transfer success. Or:

Use Pull pattern Instead of Push. Instead of transferring tokens directly, credit the amount to a claimable mapping.

Resolution: The `submitPoolUpdateBatch()` and `submitVaultUpdateBatch()` functions now implement try-catch mechanisms. In case of any failure, the transaction

just emits an event and continues with the execution by deleting the forced withdrawal request from the system. Revised commit: [322258e](#)

```
function submitPoolUpdateBatch(...) {
    //... rest of the code
    if (forcedCount > 0) {
        uint256 nextIdx = IQueueForVerifier(inclusionQueueContract).nextPoolRequestToProcessIndex();
        for (uint256 k = 0; k < forcedCount; k++) {
            uint256 queueIndex = nextIdx + k;
            (, address u, address token, uint256 amount, uint256 ts) = IQueueForVerifier(inclusionQueueContract).poolQueue(queueIndex);
            if (block.timestamp < ts + forcedDeadline) { revert Verifier_NotOverDue(); }
            uint256 bal = IPoolForVerifier(poolContract).balances(u, token);
            uint256 execAmount = bal < amount ? bal : amount;
            if (execAmount == 0) {
                emit OverdueRequestProcessed(queueIndex, u, uint8(ForcedWithdrawalSource.POOL), amount, 0, uint8(ForcedWithdrawalStatus.INSUFFICIENT_BALANCE));
            } else {
                try IPoolForVerifier(poolContract).executeOnChainWithdrawal(u, token, execAmount) {
                    emit OverdueRequestProcessed(queueIndex, u, uint8(ForcedWithdrawalSource.POOL), amount, execAmount, uint8(ForcedWithdrawalStatus.EXECUTED));
                } catch {
                    emit OverdueRequestProcessed(queueIndex, u, uint8(ForcedWithdrawalSource.POOL), amount, 0, uint8(ForcedWithdrawalStatus.FAILED));
                }
            }
        }
        //... rest of the code
    }
}
```

Evidences

PoC

Reproduce:

Steps to reproduce the issue:

PoC Code:

```
// PoC: Blacklisted user at front of queue blocks all subsequent withdrawals
```



```

function test_BlacklistedUserBlocksEntireQueue() public {
    // Users deposit into Pool
    vm.startPrank(alice, alice);
    usdc.approve(address(pool), 1000e6);
    pool.depositToken(address(usdc), 1000e6);
    vm.stopPrank();
    vm.startPrank(bob, bob);
    usdc.approve(address(pool), 1000e6);
    pool.depositToken(address(usdc), 1000e6);
    vm.stopPrank();
    vm.startPrank(charlie, charlie);
    usdc.approve(address(pool), 1000e6);
    pool.depositToken(address(usdc), 1000e6);
    vm.stopPrank();

    // Users queue forced withdrawals (FIFO order)
    // Alice queues first (index 0)
    vm.prank(alice, alice);
    queue.forceWithdrawalFromPool(1000e6);
    // Bob queues second (index 1)
    vm.prank(bob, bob);
    queue.forceWithdrawalFromPool(1000e6);
    // Charlie queues third (index 2)
    vm.prank(charlie, charlie);
    queue.forceWithdrawalFromPool(1000e6);

    // Alice gets blacklisted
    usdc.blacklist(alice);
    // Wait for forced deadline to pass
    vm.warp(block.timestamp + 1 days + 1);
    // Try escape hatch FAILS because Alice is at front
    vm.expectRevert("Recipient is blacklisted");
    verifier.processOverdueQueueItem(0); // 0 = POOL
}

```

Results:

```

Ran 1 test for test/BlacklistQueueDoS.t.sol:BlacklistQueueDoSTest
[PASS] test_BlacklistedUserBlocksEntireQueue() (gas: 656842)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 7.42ms (1.55ms C
PU time)

```

Files:

BlacklistQueueDoS.t.sol

[F-2026-14500](#) - finalizeForceWithdrawal Silently Burns User Balance When Pool Has Insufficient Token Balance - High

Description:

The `finalizeForceWithdrawal()` function in the `Pool` contract contains a vulnerability where users can permanently lose their funds if the `Pool` contract's actual token balance is insufficient to cover the withdrawal amount. When the contract balance check fails, the function silently falls through to a cross-chain withdrawal path that decrements the user's recorded balance without transferring any tokens.

The vulnerable code path is as follows:

```
function finalizeForceWithdrawal(address _token) external notContract non
Reentrant whenNotPaused {
    ForcedWithdrawalRequest memory request = forcedWithdrawalRequests[msg.
sender][_token];
    require(request.timestamp > 0, "Pool: no forced withdrawal initiated")
;
    require(
        block.timestamp >= request.timestamp + forceWithdrawDelay,
        "Pool: withdrawal delay not passed"
    );
    uint256 amount = request.amount;
    string memory destination = request.destination;
    delete forcedWithdrawalRequests[msg.sender][_token];
    // For on-chain assets, transfer tokens; for cross-chain, just emit ev
ent as proof
    if (_token.code.length > 0) {
        // Try to transfer if it's an ERC20 contract
        try IERC20(_token).balanceOf(address(this)) returns (uint256 contr
actBalance) {
            if (contractBalance >= amount) {
                balances[msg.sender][_token] -= amount;
                IERC20(_token).safeTransfer(msg.sender, amount);
                emit ForceWithdrawalFinalized(msg.sender, _token, amount,
destination, balances[msg.sender][_token]);
                return;
            }
        } catch {
            // Not an ERC20 or transfer failed
        }
    }
    // For cross-chain assets or if transfer failed, just emit event as pe
rmanent proof
    balances[msg.sender][_token] -= amount;
    emit ForceWithdrawalFinalized(msg.sender, _token, amount, destination,
```

```
balances[msg.sender][_token]);  
}
```

When `contractBalance < amount`, the function does not return early. Instead, it falls to where:

1. The user's balance is unconditionally decremented by the full amount
2. A `ForceWithdrawalFinalized` event is emitted as if the withdrawal succeeded

This creates a scenario where legitimate user funds are effectively burned—the user's recorded balance goes to zero while they receive nothing.

- Users can lose 100% of their deposited funds during force withdrawal scenarios
- Creates an unfair race where earlier withdrawers drain the pool at the expense of later ones
- The force withdrawal mechanism, designed as a safety mechanism when sequencer is inactive, becomes a trap that can cause greater user harm

Assets:

- `./Pool.sol` [zip file]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Simple
Severity:	High

Recommendations

Remediation: The `finalizeForceWithdrawal()` function should revert when there are insufficient funds for on-chain token withdrawals, rather than silently burning user balances.

Resolution: The force-withdrawal mechanism was entirely removed from the contract by eliminating the `initiateForceWithdrawal()` and `finalizeForceWithdrawal()` functions in commit `322258e`.

Evidences

PoC

Reproduce:

Steps to reproduce the issue:

1. Alice deposits 1,000 USDC into the Pool contract
2. Bob deposits 1,000 USDC into the Pool contract (Pool now holds 2,000 USDC total)
3. Sequencer applies a P/L state change granting Bob +500 USDC profit (Bob's virtual balance = 1,500 USDC)
4. Sequencer becomes inactive for longer than `sequencerInactivityThreshold` (14 days)
5. Both Alice and Bob call `initiateForceWithdrawal()` to start the escape hatch process
6. After `forceWithdrawDelay` (7 days) passes, Bob calls `finalizeForceWithdrawal()` first and receives 1,500 USDC (Pool now has 500 USDC remaining)
7. Alice calls `finalizeForceWithdrawal()` for her 1,000 USDC balance
8. Since `contractBalance` (500) < `amount` (1000), the transfer logic is skipped but her balance is still decremented to 0
9. Alice receives 0 USDC while her entire recorded balance is burned — permanent loss of funds

PoC test:

```
function test_SequencerFrontRunCancelsForceWithdrawal() public {  
    // Alice deposits  
    vm.startPrank(alice, alice);  
    usdc.approve(address(pool), 1000e6);  
    pool.depositToken(address(usdc), 1000e6);  
    vm.stopPrank();  
    // Bob deposits  
    vm.startPrank(bob, bob);  
    usdc.approve(address(pool), 1000e6);  
    pool.depositToken(address(usdc), 1000e6);  
    vm.stopPrank();  
  
    // Sequencer applies P/L update and increase Bob's balance  
    _sequencerApplyPnL(bob, 500e6);  
  
    // Wait for inactivity  
    vm.warp(block.timestamp + SEQUENCER_INACTIVITY_THRESHOLD + 1);  
    // Alice and Bob initiate force withdrawal at the same time  
    vm.prank(alice, alice);  
    pool.initiateForceWithdrawal(address(usdc), "");  
    vm.prank(bob, bob);  
    pool.initiateForceWithdrawal(address(usdc), "");
```

```

// Wait for withdraw delay time to be passed
vm.warp(block.timestamp + FORCE_WITHDRAW_DELAY + 1);

uint256 balanceInPool = pool.balances(alice, address(usdc));
uint256 usdcBalance = usdc.balanceOf(alice);
// Bob withdraws before Alice so the total balance in the Pool doesn't
// enough for Alice now
vm.prank(bob, bob);
pool.finalizeForceWithdrawal(address(usdc));
// As Pool's balance was not enough to force withdraw the full amount,
// the finalizeForceWithdrawal() function just decreased the balance b
// y the full amount without sending any tokens!
vm.prank(alice, alice);
pool.finalizeForceWithdrawal(address(usdc));

uint256 afterBalanceInPool = pool.balances(alice, address(usdc));
uint256 afterUsdcBalance = usdc.balanceOf(alice);
// Both balances are zero as shown due to vulnerability.
console.log("Current USDC balance of Alice in Pool contract:", afterBa
lanceInPool);
console.log("What it transferred to Alice after withdrawing:", afterUs
dcBalance - usdcBalance);
}

```

Results:

```

Ran 1 test for test/PoolForceWithdrawalWithRealVerifier.t.sol:PoolForceWithdr
awalWithRealVerifierTest
[PASS] test_SequencerFrontRunCancelsForceWithdrawal() (gas: 317482)
Logs:
Current USDC balance of Alice in Pool contract: 0
What it transferred to Alice after withdrawing: 0

```

Files:

PoolForceWithdrawalWithRealVerifier.t.sol

[F-2025-14443](#) - Uninitialized minWithdrawAmount Allows Zero-Amount Forced Withdrawal Requests To Block Queue Processing - Medium

Description:

The `minWithdrawAmount` state variable in `InclusionQueue.sol` is **never initialized** during contract deployment, defaulting to 0. This allows any user to queue a forced withdrawal request with amount = 0, which passes all validation checks in `InclusionQueue.sol`. However, when this request is processed, `Pool.executeOnChainWithdrawal()` explicitly requires amount > 0 and **reverts**. Due to the strict FIFO queue processing, this zero-amount request permanently blocks all subsequent forced withdrawal requests, completely disabling the escape hatch mechanism.

While `minWithdrawAmount` can be set via the `setAmount()` function, this provides no protection against the vulnerability. There is no check requiring `setAmount()` to be called before the queue becomes operational. Between deployment and the owner calling `setAmount()`, attackers can queue zero-amount requests

This means even if the owner eventually sets `minWithdrawAmount` to a proper value, the system can already be irreversibly compromised by zero-amount requests submitted during the unprotected window after deployment.

Missing initialization:

```
function initialize(address _usdcToken, address _governance, address _owner) external initializer {
    require(_usdcToken != address(0), "Queue: 0x usdc");
    require(_governance != address(0), "Queue: 0x gov");
    require(_owner != address(0), "Queue: owner 0x");
    __UUPSUpgradeable_init();
    __ReentrancyGuard_init();
    __Pausable_init(_owner);
    usdcToken = _usdcToken;
    governance = _governance;
    // minWithdrawAmount is NEVER set - defaults to 0
}
```

Therefore, insufficient validation allows zero amount in `InclusionQueue.sol`:

```
function forceWithdrawalFromPool(uint256 amount) external whenNotPaused nonReentrant notContract {
    require(amount >= minWithdrawAmount, "Queue: amount too small"); // 0
    // 0 ✓ PASSES
```

```

require(poolContract != address(0), "Queue: pool not set");
uint256 bal = IPoolForQueue(poolContract).balances(msg.sender, usdcToken);
en);
require(bal >= amount, "Queue: insufficient pool balance"); // bal >=
0 ✓ ALWAYS PASSES
// ... rest of the code
}

```

Execution reverts on zero amount in `Pool.sol`:

```

function executeOnChainWithdrawal(
    address user,
    address token,
    uint256 amount
) external onlyWM0rVRF {
    require(user != address(0), "Pool: user 0x");
    require(token != address(0), "Pool: token 0x");
    require(amount > 0, "Pool: amount = 0"); // REVERTS when amount = 0
    // ... rest of the code
}

```

FIFO enforcement causes permanent block in `Verifier.sol`:

```

function submitPoolUpdateBatch(...) external onlySequencer whenNotPaused {
    //... rest of the code
    require(forcedQueueIds[0] == nextIdx, "VRF: forced must start at
pointer");
    for (uint256 k = 0; k < lenForced; k++) {
        require(forcedQueueIds[k] == nextIdx + k, "VRF: forced not con
tiguous");
        (, address u, address token, uint256 amount, uint256 ts) =
            IQueueForVerifier(inclusionQueueContract).poolQueue(forced
QueueIds[k]);
        require(block.timestamp >= ts + forcedDeadline, "VRF: not over
due");
        uint256 bal = IPoolForVerifier(poolContract).balances(u, token
);
        if (bal >= amount) { // 0 >= 0 is TRUE, so it enters this blo
ck
            IPoolForVerifier(poolContract).executeOnChainWithdrawal(u,
token, amount);
            // REVERTS HERE - entire transaction fails

        //... rest of the code
    }
}

```

Assets:

- `./Verifier.sol` [zip file]

- ./Pool.sol [zip file]
- ./InclusionQueue.sol [zip file]

Status:

Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Initialize `minWithdrawAmount` in `initialize()` with a value greater than zero. Add explicit zero check in `forceWithdrawalFromPool()` for the `amount` parameter to be greater than zero.

Resolution: `initialize()` function is modified to initialize `minWithdrawAmount` variable with 1 in the commit `322258e`.

```
function initialize(address _usdcToken, address _governance, address _owner) external initializer {
    require(_usdcToken != address(0), Queue_Token0x());
    require(_governance != address(0), Queue_Governance0x());
    require(_owner != address(0), Queue_Owner0x());
    __UUPSUpgradeable_init();
    __ReentrancyGuard_init();
    __Pausable_init(_owner);
    usdcToken = _usdcToken;
    governance = _governance;
    minWithdrawAmount = 1;
}
```


[F-2025-14448](#) - Uncoordinated Escape Hatch Mechanisms Cause Permanent forcedWithdrawalRequests Lock When InclusionQueue Executes First - Medium

Description:

The protocol implements two separate escape hatch mechanisms to allow users to withdraw funds when the sequencer is unresponsive or censoring:

1. `InclusionQueue.forceWithdrawalFromPool(uint256 amount)`
 - Users pay a fee to queue a forced withdrawal request in the `poolQueue[]` array
 - Available anytime (doesn't require sequencer to be inactive)
 - Can be processed in two ways:
 - By sequencer: via `Verifier.submitPoolUpdateBatch()` which includes `forcedQueueIds[]` parameter
 - By anyone: via `Verifier.processOverdueQueueItem(uint8 source)` after `forcedDeadline` (24 hours) has passed — this is a permissionless escape hatch that allows any user to trigger processing of overdue queue items when the sequencer is not responding
 - Both processing paths ultimately call `Pool.executeOnChainWithdrawal()` to transfer tokens
2. `Pool.initiateForceWithdrawal(address _token, string calldata destination)`
 - Users initiate a direct forced withdrawal stored in `forcedWithdrawalRequests[user][token]`
 - Only available when sequencer has been inactive for `sequencerInactivityThreshold` (14 days)
 - User must wait `forceWithdrawDelay` (7 days) then call `Pool.finalizeForceWithdrawal(address _token)` to complete
 - Transfers tokens directly from `Pool` to user

Both mechanisms are designed to provide censorship resistance, but they operate independently with no coordination or awareness of each other.

When a user has both an active `InclusionQueue` forced withdrawal request (via `forceWithdrawalFromPool()`) and a `Pool.forcedWithdrawalRequests` entry (via `initiateForceWithdrawal()`), a critical asymmetry exists in how each path handles insufficient balances:

1. `InclusionQueue` path (processed via `Verifier.submitPoolUpdateBatch()` or `Verifier.processOverdueQueueItem()`) — Handles gracefully:

```
// Verifier.sol - Both submitPoolUpdateBatch() and processOverdueQueueItem()
use this pattern:
```

```
uint256 bal = IPoolForVerifier(poolContract).balances(u, token);
if (bal >= amount) {
    IPoolForVerifier(poolContract).executeOnChainWithdrawal(u, token, amount);
    emit OverdueRequestProcessed(forcedQueueIds[k], u, 0, amount);
} else {
    // Gracefully skips - emits event with 0 amount, queue advances
    emit OverdueRequestProcessed(forcedQueueIds[k], u, 0, 0);
}
```

2. However, `Pool` escape hatch path (via `finalizeForceWithdrawal()`) — Reverts and gets stuck:

```
// Pool.sol - finalizeForceWithdrawal()
uint256 amount = request.amount; // Uses amount recorded at initiation time
// ...
balances[msg.sender][_token] -= amount; // REVERTS on underflow if balance <
amount
```

If the `InclusionQueue` path executes first — whether triggered by the sequencer via `submitPoolUpdateBatch()` or by any user via `processOverdueQueueItem()` — and drains the user's `balances[user][token]` to zero, the subsequent call to `Pool.finalizeForceWithdrawal()` will:

1. Revert due to Solidity 0.8+ underflow protection when attempting `balances[msg.sender][_token] -= amount` (where balance is now 0 but amount is the original requested value)
2. Leave `forcedWithdrawalRequests[user][token]` intact — because the transaction reverted before reaching the delete statement
3. Block future calls to `initiateForceWithdrawal()` — because the check `forcedWithdrawalRequests[msg.sender][_token].timestamp == 0` will fail (timestamp is still set from the original request)

The user is now permanently stuck: they cannot finalize (underflow), cannot re-initiate (request exists), and cannot cancel (no cancel function exists). The only way to clear the stuck `forcedWithdrawalRequests` entry is if the sequencer calls `applyStateChanges()` with this user included,.

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple

Severity:

Medium

Recommendations**Remediation:**

Add cleanup of `forcedWithdrawalRequests` in `executeOnChainWithdrawal()` to ensure the `Pool` escape hatch state is cleared when balance is withdrawn through any path or modify `finalizeForceWithdrawal()` to use current balance instead of stored amount and delete the request before any operations that could revert:

Resolution:

This issue is implicitly resolved by removing the Pool-level force withdrawal mechanism (`initiateForceWithdrawal()` and `finalizeForceWithdrawal()`), leaving only the `InclusionQueue` -based escape hatch, which handles insufficient balances gracefully without creating stuck user state. Revised commit: `322258e` .

[F-2025-14461](#) - Fee-on-Transfer / Rebasing Tokens Break Accounting

- Medium

Description:

The `depositToken()` function assumes that the `Pool` contract receives exactly the amount of tokens specified by the user. However, this assumption does not hold for fee-on-transfer, deflationary, or rebasing ERC20 tokens.

When a user deposits such a token, the token contract may deduct a transfer fee, burn a portion of the amount, or otherwise reduce the number of tokens actually transferred to the `Pool`. Despite this, the Pool credits the user's internal balance by the full amount parameter, without verifying how many tokens were actually received.

As a result, the internal accounting can become inconsistent with the `Pool`'s real token balance. The `Pool` may record a higher balance for the user than the contract actually holds on-chain, leading to balance inflation.

This inconsistency can later surface during on-chain withdrawals, forced withdrawals, or emergency exit flows, where the Pool attempts to transfer tokens that it does not actually have. In such cases, withdrawals may revert or fail, effectively locking user funds or breaking the escape-hatch mechanism.

`Pool.sol` — `depositToken()`:

```
function depositToken(address token, uint256 amount)
    external
    whenNotPaused
    notContract
    nonReentrant
{
    require(token != address(0), "Pool: token 0x");
    require(amount > 0, "Pool: amount = 0");

    IERC20(token).safeTransferFrom(msg.sender, address(this), amount);
    balances[msg.sender][token] += amount;

    emit Deposit(msg.sender, token, amount, balances[msg.sender][token]);
}
```

The function assumes `amount` tokens are received, but does not measure the actual balance increase of the `Pool` contract.

Impact

- Internal user balances can be inflated relative to the Pool's actual token holdings
- On-chain withdrawals may revert due to insufficient contract balance
- Forced withdrawals may fail or become unfinalizable
- The Pool may become insolvent for specific tokens

Assets:

- ./Pool.sol [zip file]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation:

When processing deposits, track the actual amount received by the contract rather than trusting the input parameter. This can be achieved by measuring the token balance before and after the transfer and crediting the user with the difference.

```
uint256 balanceBefore = IERC20(token).balanceOf(address(this));
IERC20(token).safeTransferFrom(msg.sender, address(this), amount);
uint256 balanceAfter = IERC20(token).balanceOf(address(this));
uint256 receivedAmount = balanceAfter - balanceBefore;
require(receivedAmount > 0, "No tokens received");
balances[msg.sender][token] += receivedAmount;
```

Resolution:

The deposit logic was updated to account for fee-on-transfer and deflationary tokens by measuring the Pool's token balance before and after the transfer and crediting users only with the actual amount received in commit `322258e`.

```
function depositToken(uint256 amount) external whenNotPaused nonReentrant
{
    require(usdcToken != address(0), Pool_Usdc0x());
    require(amount > 0, Pool_AmountZero());
```

```

uint256 balanceBefore = IERC20(usdcToken).balanceOf(address(this));
IERC20(usdcToken).safeTransferFrom(msg.sender, address(this), amount);
uint256 balanceAfter = IERC20(usdcToken).balanceOf(address(this));
uint256 receivedAmount = balanceAfter - balanceBefore;
require(receivedAmount > 0, Pool_NoTokenReceived());
uint256 userBalanceBefore = balances[msg.sender][usdcToken];
balances[msg.sender][usdcToken] += receivedAmount;
emit Deposit(msg.sender, usdcToken, receivedAmount, balances[msg.sender][usdcToken]);
emit BalanceUpdated(msg.sender, usdcToken, int256(receivedAmount), userBalanceBefore, balances[msg.sender][usdcToken]);
}

```

[F-2025-14485](#) - Lack of Limits and Delay in Forced Withdrawal

Parameter Updates - Medium

Description:

The `InclusionQueue` contract enables the creation of forced withdrawal requests originating from the `Vault` and `Pool` contracts. These requests are later consumed by the verifier contract to finalize withdrawals. During request creation, a fee equal to the `feeAmount` variable must be paid. This fee is not deducted from the internally deposited balance, requiring the availability of additional externally held assets.

The `setAmount()` function is responsible for updating both `minWithdrawAmount` and `feeAmount`. This function is protected by the `onlyOwner` modifier. However, several design shortcomings introduce risk and negatively affect the withdrawal process.

```
function setAmount(uint256 _minWithdrawAmount, uint256 _feeAmount) external onlyOwner {
    require(_minWithdrawAmount > 0, "Queue: 0 min amount");
    minWithdrawAmount = _minWithdrawAmount;
    feeAmount = _feeAmount;
}
```

First, no reasonable upper bounds are enforced for either parameter. As a result, `minWithdrawAmount` can be set to excessively high values, potentially preventing withdrawals of smaller deposits or blocking withdrawals entirely. Similarly, `feeAmount` can be set to arbitrarily large values, requiring disproportionately high external assets to initiate a withdrawal request and effectively restricting access to deposited assets.

Second, parameter changes take effect immediately upon execution. This behavior enables front-running scenarios and situations in which withdrawal requests are submitted without awareness of updated values. In such cases, a higher-than-expected fee may be charged, or the withdrawal request may fail due to newly enforced thresholds.

```
function forceWithdrawalFromPool(uint256 amount) external whenNotPaused nonReentrant notContract {
    require(amount >= minWithdrawAmount, "Queue: amount too small");
    //...
    if (feeAmount > 0 && usdcToken != address(0)) {
        IERC20(usdcToken).safeTransferFrom(msg.sender, address(this), feeAmount);
    }
}
```

```
//...  
}
```

Assets:

- ./InclusionQueue.sol [zip file]

Status:**Fixed**

Classification**Impact Rate:** 5/5**Likelihood Rate:** 3/5**Exploitability:** Dependent**Complexity:** Simple**Severity:** **Medium**

Recommendations**Remediation:**

It is recommended to enforce reasonable upper bounds for `feeAmount` and `minWithdrawAmount`, introduce a delay or timelock mechanism for parameter updates. These measures would improve predictability, reduce front-running risk, and prevent withdrawal restrictions caused by excessively restrictive parameter values.

Resolution:

The BullBit team implemented upper bounds for `feeAmount` and `minWithdrawAmount` and a time-lock mechanism is integrated to reflect changes in commit `322258e`.

[F-2025-14487](#) - Forced Withdrawal Flow Is Not Fully Censorship-Resistant - Medium

Description:

According to the documentation, the `InclusionQueue` contract is described as a dedicated mechanism allowing on-chain withdrawal requests that cannot be ignored, intended to function as an anti-censorship tool.

A dedicated contract allowing users to register on-chain withdrawal requests that the Sequencer cannot ignore. Core anti-censorship tool.

However, based on the current implementation and design choices, the forced withdrawal flow is not fully independent and may be censored or blocked by the sequencer or privileged roles.

First, the creation of forced withdrawal requests can be restricted or rendered economically infeasible due to unbounded and immediately applied values of `feeAmount` and `minWithdrawAmount`. These parameters are controlled by a privileged role and lack reasonable upper limits. As a result, withdrawal requests for smaller balances can be prevented, or all withdrawals can be effectively blocked by setting excessively high values. Reported in [F-2025-14485](#)

Second, regardless of whether a withdrawal request is finalized through normal sequencing or through the `processOverdueQueueItem()`, successful execution remains dependent on the off-chain sequencer component. If the internal balance is reduced after request creation, the withdrawal request will not be finalized. When the internal balance stored in the balance mapping is lower than the cached request amount, the withdrawal execution path is skipped entirely. To finalize a forced withdrawal request, a delay equal to the `forcedDeadline` must elapse. The default value after deployment is 24 hours, but it can be increased up to 7 days. During this period, the likelihood of internal balance changes is increased, further raising the probability that the withdrawal request becomes unexecutable.

```
function processOverdueQueueItem(uint8 source) external nonReentrant whenNotPaused {
    // ...
    if (s == ForcedWithdrawalSource.POOL) {
        balance = IPoolForVerifier(poolContract).balances(user, token);
        if (balance >= amount) {
            IPoolForVerifier(poolContract).executeOnChainWithdrawal(user, token, amount);
            executed = true;
        }
        IQueueForVerifier(inclusionQueueContract).setPoolQueueIndex(1);
    }
}
```

```

    } else {
        balance = IVaultForVerifier(vaultContract).balances(user);
        if (balance >= amount) {
            IVaultForVerifier(vaultContract).executeOnChainWithdrawal(user, amount);
            executed = true;
        }
        IQueueForVerifier(inclusionQueueContract).setVaultQueueIndex(1);
    }
    emit OverdueRequestProcessed(idx, user, source, executed ? amount : 0);
}

function submitVaultUpdateBatch() external onlySequencer whenNotPaused {
    // ...
    uint256 bal = IVaultForVerifier(vaultContract).balances(u);
    if (bal >= amount) {
        IVaultForVerifier(vaultContract).executeOnChainWithdrawal(u, amount);
        emit OverdueRequestProcessed(forcedQueueIds[k], u, 1, amount);
    } else {
        emit OverdueRequestProcessed(forcedQueueIds[k], u, 1, 0);
    }
    // ...
}

```

Internal balances can be modified through `submitPoolUpdateBatch()` and `submitVaultUpdateBatch()` without direct user interaction, signatures, or approvals. This allows balance adjustments to occur after a forced withdrawal request has been created, preventing finalization of the request.

On top of that, the sequencer could deliberately process only a single withdrawal request per transaction—solely to appear active—while leaving the rest of the queue unprocessed. The sequencer could also selectively execute forced withdrawal requests for specific users, while effectively blocking or delaying requests from others. This further undermines the censorship-resistance guarantees of the forced withdrawal mechanism and enables discriminatory or selective access to withdrawals.

These design choices collectively result in a withdrawal mechanism that is not fully censorship-resistant. Control by privileged roles and reliance on off-chain components allow withdrawal requests to be delayed, blocked, or rendered ineffective, contradicting the stated anti-censorship guarantees.

Assets:

- `./Verifier.sol` [zip file]
- `./InclusionQueue.sol` [zip file]

Status:

Fixed

Classification

Impact Rate:	5/5
Likelihood Rate:	4/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation:	It is recommended to decouple forced withdrawals from cached balance snapshots and execute withdrawals based on the current internal balance at the time of finalization. When the balance is lower than the cached request amount, the withdrawal should succeed for the available balance rather than reverting or skipping execution.
Resolution:	The issue was fixed in commit 322258e . In the forced withdrawal flow, if a user's balance is decreased after the request is created, the remaining internal balance will be transferred instead of reverting.

[F-2025-14494](#) - InclusionQueue Forced Withdrawal Mechanism Limited to USDC - Medium

Description:

The `Pool` contract allows deposits of arbitrary ERC20 tokens by specifying the token address and deposit amount. Withdrawals are normally processed through an off-chain request submitted via a dApp, after which a centralized sequencer is responsible for including the withdrawal in the `submitPoolUpdateBatch()` function call that performs on-chain withdrawals and other state updates.

```
function depositToken(address token, uint256 amount) external whenNotPaused nonReentrant {
    require(token != address(0), "Pool: token 0x");
    require(amount > 0, "Pool: amount = 0");
    IERC20(token).safeTransferFrom(msg.sender, address(this), amount);
    balances[msg.sender][token] += amount;
    emit Deposit(msg.sender, token, amount, balances[msg.sender][token]);
}
```

To mitigate potential censorship by the dApp or sequencer, a forced withdrawal mechanism is implemented. The `Pool` contract includes a forced withdrawal flow intended to allow the creation and finalization of withdrawals without reliance on the normal sequencing process, but this flow can be altered by Sequencer - as reported in the [F-2025-14487 | Forced Withdrawal Flow Is Not Fully Censorship-Resistant](#). In parallel, the `InclusionQueue` contract provides similar functionality by allowing withdrawal requests to be queued and later finalized either by the sequencer or through a user-triggered fallback mechanism. However, the `forceWithdrawalFromPool()` function restricts forced withdrawals to the USDC token only. As a result, while the `Pool` contract permits deposits of any ERC20 token without restriction, the anti-censorship withdrawal flow supports only USDC. Tokens other than USDC cannot be withdrawn through the forced withdrawal mechanism.

```
function forceWithdrawalFromPool(uint256 amount) external whenNotPaused nonReentrant notContract {
    require(amount >= minWithdrawAmount, "Queue: amount too small");
    require(poolContract != address(0), "Queue: pool not set");
    uint256 bal = IPoolForQueue(poolContract).balances(msg.sender, usdcToken);
    ;
    require(bal >= amount, "Queue: insufficient pool balance");
    if (feeAmount > 0 && usdcToken != address(0)) {
        IERC20(usdcToken).safeTransferFrom(msg.sender, address(this), feeAmount);
    }
}
```

```

    }
    ForcedWithdrawalRequest memory req = ForcedWithdrawalRequest({
        id : requestPoolId,
        user : msg.sender,
        token : usdcToken,
        amount : amount,
        timestamp : block.timestamp
    });
    poolQueue.push(req);
    requestPoolId += 1;
    emit WithdrawalForced(req.id, msg.sender, amount, WithdrawalSource.POOL);
}

```

This design creates an inconsistency where unrestricted deposits are permitted, but anti-censorship withdrawals are limited to a single asset. Since the standard withdrawal flow remains dependent on the centralized sequencer and can be delayed or prevented, the `InclusionQueue`-based forced withdrawal mechanism represents the only censorship-resistant path for asset recovery. Limiting this mechanism to USDC prevents the recovery of other deposited assets under censorship scenarios.

Assets:

- `./InclusionQueue.sol` [zip file]

Status:

Accepted

Classification

Impact Rate:	4/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation:

It is recommended to extend the forced withdrawal functionality to support all ERC20 tokens that can be deposited into the `Pool`. The asset address should be included as a parameter in the forced withdrawal request in the `InclusionQueue` contract, and withdrawal finalization logic should operate generically across supported tokens. This alignment would ensure that the anti-censorship withdrawal mechanism provides consistent asset recovery guarantees for all deposited assets and matches the flexibility of the deposit flow.

Resolution:

The issue was accepted. In commit `e665fb628fd70fc9dd01b04067f7977ec08874e4`, the tokens allowed for deposit via the `depositToken()` function were narrowed to only the address stored in the `usdcToken` storage variable. However, the `Pool` contract still supports storage for multiple tokens. Non-USDC tokens can be deposited via the cross-chain flow, and their balances can be increased through the `Verifier` contract executing `applyStateChanges()`.

The client team provided the following statement regarding this issue:

We acknowledge the inconsistency highlighted in issue F-2025-14494. However, rather than expanding the InclusionQueue to support arbitrary tokens (which would introduce significant complexity and potential risks at this stage), we will restrict the Deposit flow to align with our current withdrawal capabilities.

Remediation Plan:

We will modify the `depositToken` function in the `Pool` contract to only accept the designated USDC token. Any attempt to deposit other ERC20 tokens will revert.

Rationale & Architectural Context:

1. **Settlement Asset:** Our system is designed with USDC as the primary On-chain Settlement Asset on Base.
2. **Cross-Chain Architecture:** For non-USDC assets (e.g., ETH, BTC), we are implementing a "Vault - Messenger - Ledger" architecture powered by the LayerZero OApp Standard.
 - Native assets are locked in Vaults on their source chains (e.g., Ethereum, BNB Chain).
 - The Pool contract on Base acts as a Ledger, recording credits via cross-chain messaging, rather than directly holding the physical ERC20 tokens of those assets on Base.
1. **Preventing Stuck Funds:** Allowing direct deposits of arbitrary ERC20 tokens (such as WETH on Base) into the Pool is technically an architectural anti-pattern, as it bypasses the cross-chain settlement logic. Restricting deposits to USDC ensures that users cannot lock unsupported assets into the contract.

Future Roadmap:

Once the LayerZero OApp integration is fully deployed, the InclusionQueue will be upgraded to support triggering cross-chain messages (forcing a debit on Base to unlock funds on the Source Chain). For the current release, limiting deposits to USDC effectively eliminates the risk of stuck funds due to censorship.

[F-2026-14499](#) - Cross-Chain Deposit Credits Not Backed by Locked Assets - Medium

Description:

The `Pool` contract manages on-chain internal balances for multiple ERC20 tokens. In addition, cross-chain deposit and withdrawal functions are implemented and are intended to be invoked only by other system contracts, such as the `Verifier` and `WithdrawalManager`.

According to the documentation, a cross-chain deposit is initiated off-chain, after which assets are transferred to a generated address on the source chain. Once the transfer is detected, the `creditCrossChainDeposit()` function is expected to be executed by including the credit operation in a `submitPoolUpdateBatch()` call. This function increases the internal balance of a user by an arbitrary amount without transferring or locking the equivalent ERC20 tokens within the `Pool` contract.

```
function creditCrossChainDeposit(address user, address token,
                                uint256 amount) external onlyVerifier {
    require(user != address(0), "Pool: user 0x");
    require(token != address(0), "Pool: token 0x");
    require(amount > 0, "Pool: amount = 0");
    balances[user][token] += amount;
    emit Deposit(user, token, amount, balances[user][token]);
}
```

As a result, the aggregate of internal balances recorded in the `Pool` contract can exceed the amount of ERC20 tokens actually held by the contract. This imbalance may affect availability of assets for other participants, as withdrawals may fail due to insufficient token balances in the `Pool` contract. Assets credited through the cross-chain flow can be withdrawn on the destination chain, while other balances may become unredeemable because the `Pool` contract does not hold sufficient tokens.

In addition, the `creditCrossChainDeposit()` function enables the actor controlling the sequencer to arbitrarily increase the internal balance of any address. This allows inflation of internal balances followed by withdrawal of assets held by the `Pool` contract, resulting in complete depletion of available tokens.

The withdrawal process requires a user signature and reduces the internal balance within the `Pool` contract. However, the actual transfer of ERC20 tokens is executed by an off-chain component that is outside the scope of this audit. If this off-chain component fails or behaves incorrectly, there is no on-chain guarantee that the corresponding assets will be transferred, further weakening withdrawal safety guarantees.

```
function executeCrossChainWithdrawal(address user, address token, uint256 amount,
                                string calldata destination) external onlyWithdrawalManager {
    require(user != address(0), "Pool: user 0x");
    require(token != address(0), "Pool: token 0x");
    require(amount > 0, "Pool: amount = 0");
    require(bytes(destination).length != 0, "Pool: dest empty");

    uint256 bal = balances[user][token];
    require(bal >= amount, "Pool: insufficient");
    balances[user][token] = bal - amount;
    emit CrossChainWithdrawal(user, token, amount, destination, balances[user][token]);
}
```

Assets:

- ./Pool.sol [zip file]

Status:

Accepted

Classification

Impact Rate: 4/5

Likelihood Rate: 4/5

Exploitability: Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation:

It is recommended to either:

- remove the cross-chain deposit and withdrawal flow entirely and require assets to be bridged externally before using the single-chain deposit and withdrawal mechanism,
- or enforce strict asset backing by transferring and locking the corresponding ERC20 tokens within the `Pool` contract during execution of `creditCrossChainDeposit()`. If the cross-chain flow is retained, the credited amount should be matched by an on-chain `safeTransferFrom()` of the same token amount to the `Pool` contract to ensure internal balances remain fully backed by actual assets. This approach would mitigate the issue; however, the cross-chain flows

would still remain heavily centralized and dependent on the off-chain component.

Resolution:

The associated risks are acknowledged and accepted as a known design trade-off, and no changes are applied. The client confirmed that the credit-based cross-chain deposit model is an intentional design choice, where the Pool acts as a settlement ledger and asset backing is enforced off-chain via source-chain vaults and a trusted sequencer.

[F-2026-14505](#) - Inconsistent Asset Accounting Between Internal Balances and Contract Holdings - Medium

Description:

USDC can be deposited into the `Vault` contract, and arbitrary ERC20 tokens can be deposited into the `Pool` contract to obtain internal credit used for trading actions in the off-chain application. In both contracts, deposits are performed through the `deposit()` or `depositToken()` functions, which follow similar logic. These functions perform basic safety checks, transfer tokens into the contract, and update the internal `balances` mapping accordingly.

```
// Vault.sol
function deposit(uint256 amount) external notContract nonReentrant whenNotPaused {
    require(amount > 0, "Vault: amount = 0");
    IERC20(usdcToken).safeTransferFrom(msg.sender, address(this), amount);
    balances[msg.sender] += amount;
    emit Deposit(msg.sender, amount, balances[msg.sender]);
}
```

```
// Pool.sol
function depositToken(address token,
    uint256 amount) external whenNotPaused notContract nonReentrant {
    require(token != address(0), "Pool: token 0x");
    require(amount > 0, "Pool: amount = 0");
    IERC20(token).safeTransferFrom(msg.sender, address(this), amount);
    balances[msg.sender][token] += amount;
    emit Deposit(msg.sender, token, amount, balances[msg.sender][token]);
}
```

When internal balances are modified off-chain, the updated state is later reflected on-chain. The sequencer aggregates user balance deltas together with other actions, such as withdrawal requests and cross-chain operations, and submits them to the `Verifier` contract via `submitPoolUpdateBatch()` or `submitVaultUpdateBatch()`, depending on the context. In both cases, internal balances are updated through the `applyStateChanges()` function, which adds or subtracts values from the `balances` mapping.

However, the `applyStateChanges()` function does not enforce that additional ERC20 tokens are transferred into the `Pool` or `Vault` contracts when the aggregate sum of internal balances for a given token increases. As a

result, internal accounting can diverge from the actual token balances held by the contracts.

```
function applyStateChanges(address token, address[] calldata users,
                           uint256[] calldata deltas) external onlyVerifier {
    require(token != address(0), "Pool: token 0x");
    require(users.length == deltas.length, "Pool: length mismatch");

    for (uint256 i = 0; i < users.length; i++) {
        uint256 d = deltas[i];
        if (d == 0) continue;

        address u = users[i];
        if (d > 0) {
            balances[u][token] += uint256(d);
        } else {
            uint256 dec = uint256(-d);
            uint256 bal = balances[u][token];
            require(bal >= dec, "Pool: negative balance");
            balances[u][token] = bal - dec;
        }

        // Cancel any pending forced withdrawal request
        delete forcedWithdrawalRequests[u][token];
    }
}
```

This behavior enables multiple risk scenarios. An actor controlling the sequencer can arbitrarily increase the internal balance of a selected address and subsequently withdraw assets held by the **Pool** or **Vault** contracts. This risk is amplified by the fact that standard withdrawal flows are also sequencer-controlled, while forced withdrawal mechanisms are subject to timing constraints, limiting the ability to react to suspicious activity.

Additionally, situations may arise in which participants are unable to withdraw their assets due to insufficient ERC20 token balances in the contracts. If the total sum of internal balances exceeds the actual contract balance for a given token, later withdrawals may fail, creating an unfair execution order in which earlier withdrawals succeed at the expense of subsequent ones.

Assets:

- ./Pool.sol [zip file]
- ./Vault.sol [zip file]

Status:

Accepted

Classification

Impact Rate:	4/5
Likelihood Rate:	4/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: It is recommended to enforce consistency between internal balances and actual ERC20 token holdings within both Pool and Vault contracts. The aggregate sum of internal balances should be explicitly tracked using an additional state variable in the Pool contract and a per-token mapping in the Vault contract. These values should be updated whenever the `balances` mapping is modified and validated within `applyStateChanges()`. When the net balance delta is positive, the corresponding amount of tokens should be transferred into the contract before state updates are finalized.

Resolution: The issue was accepted by the Client team with the following statement:
To clarify the accounting discrepancies noted in the report, please distinguish the specific business roles of each contract:

Pool Contract (Trader Collateral & Settlement Ledger):

- **User Base:** General Retail Traders.
- **Assets:** It holds USDC (as trading collateral) and acts as the Settlement Ledger for cross-chain assets (BTC, ETH...).
- **Note:** As mentioned, cross-chain assets are backed on the Source Chain, while the Pool records the credit.

Vault Contract (LP Liquidity for Perps):

- **User Base:** Liquidity Providers (LPs/Market Makers) only.
- **Assets:** It holds strictly USDC.
- **Function:** This follows a model similar to **Hyperliquid's HyperVault**, where LPs deposit USDC to provide counterparty liquidity specifically for Perpetual Futures.
- **Scope:** The Vault is NOT involved in Spot trading or cross-chain asset custody.

Conclusion: Because the Vault is restricted to USDC-based Perp liquidity, while the Pool acts as a multi-asset ledger, their internal accounting logic is intentionally separated to isolate risk profiles."

Evidences

PoC

Reproduce:

```
function test_internal_balance_does_not_equal_contract_balance() public {
    uint256 user1Balance = 100e6;
    usdc.mint(user1, user1Balance);

    vm.startPrank(user1, user1);
    usdc.approve(address(vault), user1Balance);
    vault.deposit(user1Balance);
    vm.stopPrank();

    assertEq(vault.balances(user1), user1Balance);
    assertEq(vault.balances(user2), 0);
    assertEq(usdc.balanceOf(address(vault)), user1Balance);

    console.log("vault.balances(user1):", vault.balances(user1));
;
    console.log("vault.balances(user2):", vault.balances(user2));
;
    console.log("usdc.balanceOf(address(vault)):", usdc.balanceOf(address(vault)));

    address[] memory users = new address[](1);
    users[0] = user2;
    int256[] memory deltas = new int256[](1);
    deltas[0] = int256(100e6);

    // simply call, it skips the submitVaultUpdateBatch() for simplicity
    vm.prank(address(verifier), address(verifier));
    vault.applyStateChanges(users, deltas);

    assertEq(vault.balances(user1), user1Balance);
    assertEq(vault.balances(user2), 100e6);
    assertEq(usdc.balanceOf(address(vault)), user1Balance);

    console.log("-----");
    console.log("After applying state changes");
    console.log("vault.balances(user1):", vault.balances(user1));
;
    console.log("vault.balances(user2):", vault.balances(user2));
;
    console.log("usdc.balanceOf(address(vault)):", usdc.balanceOf(address(vault)));
}
}
```

Results:

```
[PASS] test_internal_balance_does_not_equal_contract_balance() (gas: 167059)
```

Logs:

```
vault.balances(user1):      100000000  
vault.balances(user2):      0  
usdc.balanceOf(address(vault)): 100000000
```

After applying state changes

```
vault.balances(user1):      100000000  
vault.balances(user2):      100000000  
usdc.balanceOf(address(vault)): 100000000
```

[F-2026-14523](#) - Pending Force Withdrawal Requests Removed On Balance Update - Medium

Description:

The `Pool` and `Vault` contracts implement a force withdrawal flow that enables withdrawal of tokens when the off-chain Sequencer is inactive. The Sequencer is responsible for normal withdrawal execution, while the force withdrawal flow provides a fallback mechanism when the Sequencer is inactive for a period equal to the `sequencerInactivityThreshold` (default 14 days).

During `initiateForceWithdrawal()`, a `ForcedWithdrawalRequest` struct is created and stored in the `forcedWithdrawalRequests` mapping using `msg.sender` as the key. After a delay equal to `forceWithdrawDelay` (default 7 days), `finalizeForceWithdrawal()` allows tokens to be transferred to the user based on the cached balance at the time of the initiation.

```
function initiateForceWithdrawal() external notContract nonReentrant whenNotPaused {
    require(verifierContract != address(0), "Vault: verifier not set");
    // Check if sequencer is inactive
    uint256 lastUpdate = IVerifierVault(verifierContract).lastVaultUpdateTimeStamp();
    require(block.timestamp > lastUpdate + sequencerInactivityThreshold,
        "Vault: sequencer is still active");

    uint256 bal = balances[msg.sender];
    require(bal > 0, "Vault: no balance to withdraw");
    require(forcedWithdrawalRequests[msg.sender].timestamp == 0,
        "Vault: forced withdrawal already initiated");

    forcedWithdrawalRequests[msg.sender] =
        ForcedWithdrawalRequest({amount : bal, timestamp : block.timestamp});

    emit ForceWithdrawalInitiated(msg.sender, bal, block.timestamp);
}

function finalizeForceWithdrawal() external notContract nonReentrant whenNotPaused {
    ForcedWithdrawalRequest memory request = forcedWithdrawalRequests[msg.sender];
    require(request.timestamp > 0, "Vault: no forced withdrawal initiated");
    require(block.timestamp >= request.timestamp + forceWithdrawDelay,
        "Vault: withdrawal delay not passed");
    uint256 amount = request.amount;
    delete forcedWithdrawalRequests[msg.sender];
}
```

```

balances[msg.sender] -= amount;
IERC20(usdcToken).safeTransfer(msg.sender, amount);
emit ForceWithdrawalFinalized(msg.sender, amount, balances[msg.sender]);
}

```

Between initiation and finalization, user balances can be updated off-chain and applied on-chain via `applyStateChanges()`. If the user balance changes, even by 1 wei, the `ForcedWithdrawalRequest` struct is removed. This behavior prevents execution of a withdrawal request when the user balance is insufficient. However, if the user balance increases, removing the request lowers user experience and requires creation of a new request with additional delay.

```

function applyStateChanges(address[] calldata users,
                          uint256[] calldata deltas) external onlyVerifier {
    require(users.length == deltas.length, "Vault: length mismatch");

    for (uint256 i = 0; i < users.length; i++) {
        uint256 d = deltas[i];
        if (d == 0) continue;

        address u = users[i];
        if (d > 0) {
            balances[u] += uint256(d);
        } else {
            uint256 dec = uint256(-d);
            uint256 bal = balances[u];
            require(bal >= dec, "Vault: negative balance");
            balances[u] = bal - dec;
        }
        // Cancel any pending forced withdrawal request
        delete forcedWithdrawalRequests[u];
    }
}

```

The current mechanism allows the Sequencer to arbitrarily affect the forced withdrawal flow. Even minimal balance changes can remove pending requests, reducing the effectiveness of this mechanism as a censorship-resistant tool.

Assets:

- ./Pool.sol [zip file]
- ./Vault.sol [zip file]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	4/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: It is recommended to remove the deletion of `ForcedWithdrawalRequest` entries within `applyStateChanges()`. Additionally, `finalizeForceWithdrawal()` should withdraw the user's current balance rather than the cached balance from `initiateForceWithdrawal()`. This ensures that pending requests are not removed if balances change, improving user experience and reinforcing the censorship-resistant properties of the flow.

Resolution: The issue was fixed in commit `e665fb628fd70fc9dd01b04067f7977ec08874e4`. The `initiateForceWithdrawal()` and `finalizeForceWithdrawal()` functions were removed, along with the deletion of the `forcedWithdrawalRequests` mapping in the `applyStateChanges()` function, removing the root cause.

[F-2025-14396](#) - Inconsistent System Configuration Due to Non-Atomic Governance Updates - Low

Description:

The protocol relies on multiple interdependent contracts (governance, pool, vault, withdrawal manager, verifier, and inclusion queue) that must share a consistent system configuration (governance address, verifier address, pool/vault routing, etc.) in order to operate safely.

However, governance updates to these components are performed via multiple independent transactions, each updating a single contract at a time. There is no mechanism ensuring that all related contracts are updated atomically as part of a single system-wide configuration change.

```
function setQueueInfo(address newGov, address verifier, address pool, address vault) external onlyOwner nonReentrant {
    require(inclusionQueueContract != address(0), "Gov: queue not set");
    IQueueGovernance(inclusionQueueContract).setInfo(newGov, verifier, pool, vault);
}

function setPoolInfo(address newGov, address verifier, address _wm) external onlyOwner nonReentrant {
    require(poolContract != address(0), "Gov: pool not set");
    IPoolGovernance(poolContract).setInfo(newGov, verifier, _wm);
}

function setVaultInfo(address newGov, address verifier, address _wm) external onlyOwner nonReentrant {
    require(vaultContract != address(0), "Gov: vault not set");
    IVaultGovernance(vaultContract).setInfo(newGov, verifier, _wm);
}

function setVerifierInfo(address _gov, address _sequencer, address _pool, address _vault, address _wm, address _queue) external onlyOwner nonReentrant {
    require(verifierContract != address(0), "Gov: vrf not set");
    IVerifierGovernance(verifierContract).setInfo(_gov, _sequencer, _pool, _vault, _wm, _queue);
}

function setWMInfo(address _gov, address _verifier, address _pool, address _vault) external onlyOwner nonReentrant {
    require(withdrawalManagerContract != address(0), "Gov: wm not set");
    IWithdrawalManagerGovernance(withdrawalManagerContract).setInfo(_gov, _verifier, _pool, _vault);
}
```

As a result, the protocol can temporarily enter an inconsistent configuration state, where different components reference different governance addresses, verifiers, pools, vaults, or withdrawal managers.

During this intermediate state, user-facing or operator-facing functions may still be callable, even though the system no longer agrees on critical trust assumptions.

Depending on which component is updated first, this inconsistency can lead to:

- Withdrawal or queue processing failures due to mismatched contract expectations
- Requests being marked as processed in one component but rejected or ignored in another
- Stuck or skipped queue entries requiring manual governance intervention
- Temporary authorization mismatches, where one contract accepts an action that another contract rejects
- Increased risk during verifier or governance migrations, especially when off-chain signatures are involved

Assets:

- ./Governance.sol [zip file]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:	Introduce a single governance function that updates all interdependent contracts within the same transaction. Ensure that if any update fails, the entire configuration change reverts.
Resolution:	The current code has been refactored to use atomic system-wide updates in commit <code>322258e</code> . All configuration functions now call <code>_setSystemConfig()</code> which updates all modules in a single transaction.

[F-2025-14397](#) - Non-Refundable Fee Loss and Queue Spam in InclusionQueue Contract - Low

Description:

Fees are taken upfront when a user submits a forced-withdraw request. The actual balance check occurs later when the `Verifier` processes the queue. If the user's balance is insufficient at execution time (e.g., due to P/L changes or multiple queued requests), the withdrawal simply fails—but the fee is never refunded.

Additionally, there is no duplicate-request prevention, so a user can submit many requests totaling more than their balance, paying multiple fees but getting only one (or zero) successful withdrawal. Failed requests also bloat the queue, consuming storage and gas.

```
function forceWithdrawalFromPool(uint256 amount) external whenNotPaused n
onReentrant notContract {
    //.. rest of the code
    uint256 bal = IPoolForQueue(poolContract).balances(msg.sender, usdcTok
en);
    require(bal >= amount, "Queue: insufficient pool balance");
    if (feeAmount > 0 && usdcToken != address(0)) {
        IERC20(usdcToken).safeTransferFrom(msg.sender, address(this), feeA
mount);
    }
    ForcedWithdrawalRequest memory req = ForcedWithdrawalRequest({
        id: requestPoolId,
        user: msg.sender,
        token: usdcToken,
        amount: amount,
        timestamp: block.timestamp
    });
    poolQueue.push(req);
    //.. rest of the code
}

function forceWithdrawalFromVault(uint256 amount) external whenNotPaused n
onReentrant notContract {
    //.. rest of the code
    uint256 bal = IVaultForQueue(vaultContract).balances(msg.sender);
    require(bal >= amount, "Queue: insufficient vault balance");
    if (feeAmount > 0 && usdcToken != address(0)) {
        IERC20(usdcToken).safeTransferFrom(msg.sender, address(this), feeA
mount);
    }
}
```

```

    ForcedWithdrawalRequest memory req = ForcedWithdrawalRequest({
        id: requestVaultId,
        user: msg.sender,
        token: usdcToken,
        amount: amount,
        timestamp: block.timestamp
    });
    vaultQueue.push(req);
    //... rest of the code
}

```

When the queue is processed via `processOverdueQueueItem()`, these doomed requests still consume gas for whoever triggers processing, effectively making others pay gas just to clear failed entries.

```

function processOverdueQueueItem(uint8 source) external nonReentrant whenNotPaused {
    //... rest of the code
    // Processing later; if balance insufficient, emits and moves on (Verifier.sol)
    bool executed = false;
    if (s == ForcedWithdrawalSource.POOL) {
        balance = IPoolForVerifier(poolContract).balances(user, token);
        if (balance >= amount) {
            IPoolForVerifier(poolContract).executeOnChainWithdrawal(user,
            token, amount);
            executed = true;
        }
        IQueueForVerifier(inclusionQueueContract).setPoolQueueIndex(1);
    }
}

```

Impact:

- Users can lose significant fees for failed withdrawals.
- The queue can be spammed with un-executable requests (especially if the fee amount is small or zero), wasting gas and storage.
- Anyone calling `processOverdueQueueItem()` bears gas costs to iterate through and clear un-executable requests.

Assets:

- `./InclusionQueue.sol` [zip file]

Status:

Accepted

Classification

Impact Rate:

2/5

Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

- Prevent duplicate/pending requests per user+token and/or ensure total pending amount $\leq$ current balance.
- Refund fees when execution fails, or charge fees only on successful execution.
- Add a cleanup mechanism for failed/processed requests to prevent queue bloat.
- Add explicit amount > 0 guard and initialize `minWithdrawAmount` in `initialize()`.

Resolution:

Duplicate request prevention is now implemented (one pending request per user per source), along with `amount > 0` validation and `minWithdrawAmount` initialization. However, fees are still taken upfront with no refund mechanism if execution fails. Users lose their queue fee when:

- Their recorded balance becomes insufficient due to P/L adjustments between submission and execution,
- The Pool/Vault contract itself lacks sufficient USDC to fulfill the withdrawal (contract insolvency),
- The transfer fails for external reasons (e.g., blacklisted address).

In all cases, the sequencer's batch functions (`submitPoolUpdateBatch` / `submitVaultUpdateBatch`) process and delete the forced withdrawal request from the queue regardless of execution outcome—emitting `INSUFFICIENT_BALANCE` or `FAILED` status—while the user's fee remains non-refundable. The BullBit team confirmed this is intentional to prevent head-of-line blocking and ensure queue liveness; if failed requests were not consumed, a single unprocessable request would block all subsequent users.

Additionally, the queue arrays grow indefinitely without cleanup. Revised commit: `322258e`.

[F-2025-14452](#) - No Signature Expiry/Deadline - Low

Description:

The EIP-712 signature structures used for withdrawal requests in `WithdrawalManager` do not include a deadline or expiry parameter. Once a user signs a withdrawal request, that signature remains valid indefinitely until either:

1. The sequencer submits it for processing, or
2. The user's nonce is incremented by another withdrawal

```
// WithdrawalManager.sol - Current typehashes (no deadline)

bytes32 private constant _POOL_ON_CHAIN_WITHDRAW_TYPEHASH =
    keccak256("PoolOnChainWithdraw(address user,address token,uint256 amount,u
int256 nonce)");

bytes32 private constant _POOL_CROSS_CHAIN_WITHDRAW_TYPEHASH =
    keccak256("PoolCrossChainWithdraw(address user,address token,uint256 amoun
t,uint256 nonce,bytes32 destinationHash)");

bytes32 private constant _VAULT_WITHDRAW_TYPEHASH =
    keccak256("VaultWithdraw(address user,uint256 amount,uint256 nonce)");
```

This design grants the sequencer complete control over when to execute a user's signed withdrawal, with no time-bound constraint. The user cannot revoke or expire a signed request.

Assets:

- `./WithdrawalManager.sol` [zip file]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

Add deadline parameter to all withdrawal signatures:

```
// Updated typehash
bytes32 private constant _POOL_ON_CHAIN_WITHDRAW_TYPEHASH =
    keccak256("PoolOnChainWithdraw(address user,address token,uint256 amount,u
int256 nonce,uint256 deadline)");
function processOnChainPoolWithdrawal(
    address user,
    address token,
    uint256 amount,
    uint256 nonce,
    uint256 deadline, // Add deadline parameter
    bytes calldata signature
) external onlyVerifier {
    require(block.timestamp <= deadline, "WM: signature expired"); // Add che
ck

    (bool validSign, , ) = verifyOnChainPoolWithdrawalSignature(
        user, token, amount, nonce, deadline, signature
    );
    require(validSign, "WM: invalid signature");
    // ...
}
```

Apply same pattern to:

- `processCrossChainPoolWithdrawal()`
- `processVaultWithdrawal()`
- All corresponding verify functions and hash helpers

Resolution:

The issue has been fully fixed in commit `322258e`. All withdrawal signature typehashes now include a `deadline` parameter, and all processing functions validate that signatures have not expired.

For instance, added `deadline` validation in processing functions:

```
function processOnChainPoolWithdrawal(
    address user,
    address token,
    uint256 amount,
    uint256 nonce,
    uint256 deadline, // Added
    bytes calldata signature
) external onlyVerifier {
    require(deadline != 0, WM_DeadlineZero());
    require(block.timestamp <= deadline, WM_SignatureExpired()); // Added
```



```
// ... rest of logic  
}
```

[F-2025-14462](#) - notContract Modifier Is Bypassable and Breaks Modern Wallet Support - Low

Description:

`Pool`, `InclusionQueue` and `Vault` contracts rely on a `notContract` modifier intended to restrict certain functions to Externally Owned Accounts (EOAs) only:

```
modifier notContract() {  
    require(!_isContract(msg.sender), "Contract not allowed");  
    require(msg.sender == tx.origin, "Proxy contract not allowed");  
    _;  
}
```

This modifier attempts to detect and block contract-based interactions by checking the code size of `msg.sender` and comparing `msg.sender` with `tx.origin`. However, this approach is not a reliable security boundary, is trivially bypassable, and introduces significant compatibility issues with modern wallet infrastructure.

During contract deployment, a contract's code size is zero until construction completes. As a result, calls made from within a contract constructor will pass the `_isContract()` check:

```
function _isContract(address _addr) internal view returns (bool) {  
    uint256 size;  
    assembly {  
        size := extcodesize(_addr)  
    }  
    return size > 0;  
}
```

Compatibility impact:

The `notContract` modifier breaks support for legitimate and widely used wallet types, including:

- Multisig wallets (e.g., Gnosis Safe)
- Smart contract wallets
- Account abstraction wallets (ERC-4337)
- Relayers and meta-transaction systems
- Future wallet standards relying on contract-based execution

As a result, users relying on these wallets are unable to:

- Deposit funds
- Initiate forced withdrawals

- Interact with the protocol during emergency scenarios

This significantly limits protocol accessibility and can lock funds for users who do not use EOAs.

Security impact:

- The modifier can be bypassed, providing no real protection against adversarial contract interactions
- Legitimate users may be blocked from critical protocol operations
- The system relies on a false assumption that EOAs are inherently safer than contracts

Assets:

- ./Pool.sol [zip file]
- ./InclusionQueue.sol [zip file]
- ./Vault.sol [zip file]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation: Remove the `notContract` modifier entirely and avoid attempting to distinguish EOAs from contracts at the protocol level to improve compatibility.

Resolution: `notContract` modifier is removed entirely from the `Pool` contract in commit `322258e`.

[F-2025-14469](#) - Use of `transfer()` Instead of `call()` to Send Native Assets - Low

Description:

In the `InclusionQueue` contract, the `withdrawMain()` function uses built-in `transfer()` function for withdrawal of native tokens to the provided address. However, if the beneficiary is a smart contract (e.g. multisig wallet), they might not be able to receive these native tokens due to the gas limit associated with the transfer function in Solidity.

```
function withdrawMain(uint256 amount, address to) external onlyOwner {  
    payable(to).transfer(amount);  
}
```

The `transfer()` function was commonly used in earlier versions of Solidity for its simplicity and automatic reentrancy protection. However, it was identified as potentially problematic due to its fixed gas limit of `2300`.

The usage of `transfer()` function can lead to unintended function call revert when the receiving contract's `receive()` or `fallback()` functions require more than `2300` Gas for processing.

Assets:

- `./InclusionQueue.sol` [zip file]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation:

Consider using built-in `call()` function instead of `transfer()` to transfer native assets. This method does not impose a gas limit, it provides greater flexibility and compatibility with contracts having more complex business logic upon receiving the native tokens. When working with then `call()`

function ensure that its execution is successful by checking the returned boolean value.

Resolution:

The issue was fixed in commit [322258e](#). The `transfer()` function was replaced with a low-level call to perform the transfer of native tokens.

[F-2025-14463](#) - Missing Basic Validation for Forced-Withdrawal

Destination - Info

Description:

The `initiateForceWithdrawal()` function in `Pool` accepts a `destination` parameter intended to represent the withdrawal destination for cross-chain or off-chain processing. However, this parameter is not subject to basic validation before being stored on-chain.

Specifically, the contract allows:

- Empty destination strings
- Arbitrary, unbounded string values
- Invalid or malformed destination identifiers

As a result, forced-withdrawal requests can be created with meaningless or unusable destination data, which may complicate off-chain processing, monitoring, or dispute resolution during emergency exit scenarios.

```
function initiateForceWithdrawal(address _token, string calldata destination)
    external
    notContract
    nonReentrant
    whenNotPaused
{
    ...
    forcedWithdrawalRequests[msg.sender][_token] = ForcedWithdrawalRequest({
        token: _token,
        amount: userBalance,
        timestamp: block.timestamp,
        destination: destination
    });
    ...
}
```

No checks are performed to ensure that destination is non-empty, correctly formatted, or within a reasonable length.

Impact

- Forced-withdrawal records may contain invalid or unusable destination data
- Off-chain processors may be unable to execute or reconcile withdrawals
- Emergency exit flows become more error-prone

Assets:

- `./Pool.sol` [zip file]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Introduce minimal validation for the destination parameter, such as:

- Requiring non-empty values
- Enforcing a reasonable maximum length
- Optionally validating expected formats if the destination follows a known schema

Resolution: The issue was fixed in commit [322258e](#). The affected function was removed and does not exist in the mentioned commit.

[F-2025-14468](#) - Sequential Queue Processing Forces Users to Pay for Others' Withdrawals - Info

Description:

The `processOverdueQueueItem()` function implements a strict FIFO (First-In-First-Out) queue processing mechanism that only allows processing the next pending request in the queue. Users cannot selectively process their own overdue request if there are other requests ahead of them in the queue.

```
function processOverdueQueueItem(uint8 source) external nonReentrant when
NotPaused {
    require(source <= uint8(ForcedWithdrawalSource.VAULT), "VRF: invalid s
ource");
    ForcedWithdrawalSource s = ForcedWithdrawalSource(source);
    uint256 idx;
    address user;
    address token; // only for Pool
    uint256 amount;
    uint256 timestamp;
    if (s == ForcedWithdrawalSource.POOL) {
        idx = IQueueForVerifier(inclusionQueueContract).nextPoolRequestToP
rocessIndex();
        (, user, token, amount, timestamp) = IQueueForVerifier(inclusionQu
eueContract).poolQueue(idx);
    } else {
        idx = IQueueForVerifier(inclusionQueueContract).nextVaultRequestTo
ProcessIndex();
        (, user, , amount, timestamp) = IQueueForVerifier(inclusionQueueCo
ntract).vaultQueue(idx);
    }

    // ... rest of the code
}
```

The function:

1. Retrieves `nextPoolRequestToProcessIndex` - always the first unprocessed request
2. Processes only that request
3. Increments the index by 1

There is no way to specify which request ID to process. Therefore, users cannot process their own specific overdue withdrawal request and must pay gas costs to process all preceding requests in the queue first, creating an unfair cost burden where later users subsidize earlier users' withdrawals, and enabling griefing attacks where an attacker can force

victims to pay substantial gas fees by flooding the queue with minimal-cost requests.

Assets:

- ./Verifier.sol [zip file]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Modify the function to accept a `requestId` parameter while still validating the request is overdue.

Resolution: The `processOverdueQueueItem()` function now allows users to specify a request ID to withdraw their specific deposits in commit `322258e`.

[F-2025-14477](#) - Missing Event Emissions for Important State

Changes - Info

Description:

Multiple setter and administrative functions across the contracts in scope perform state modifications without emitting events to signal these changes. Event emission is a widely accepted best practice in Solidity smart contract development, as it improves transparency, off-chain indexing, and state observability.

The following functions perform state-changing operations without emitting corresponding events:

- `InclusionQueue`: `setAmount()`, `withdrawToken()`, `withdrawMain()`
- `Pool`: `setForcedInfo()`
- `Vault`: `setForcedInfo()`
- `Verifier`: `setForcedInfo()`

Each of these functions affects contract configuration, control flow, or user-related state. The absence of event emissions makes it difficult for off-chain services, such as indexers, frontends, and monitoring tools, to reliably track when significant protocol actions occur.

Assets:

- `./Verifier.sol` [zip file]
- `./Pool.sol` [zip file]
- `./InclusionQueue.sol` [zip file]
- `./Vault.sol` [zip file]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

It is recommended to emit clearly defined events for all mentioned functions.

Resolution:

The issue was fixed in commit [322258e](#). Event emission was added to the mentioned functions.

F-2025-14481 - Redundant forcedQueueIds Array Parameter - Info

Description:

The `forcedQueueIds` array parameter in both `submitPoolUpdateBatch()` and `submitVaultUpdateBatch()` is entirely redundant. The validation logic enforces that this array must be a strictly contiguous sequence starting from the current queue pointer, making the array's contents fully deterministic and predictable.

The code enforces two constraints:

```
function submitVaultUpdateBatch(
    // ... other parameters
    uint256[] calldata forcedQueueIds
) external onlySequencer whenNotPaused {
    // ... rest of the code

    // 2) forced withdraws from queue (VAULT)
    uint256 lenForced = forcedQueueIds.length;
    if (lenForced > 0) {
        require(lenForced <= maxForcedPerBatch, "VRF: too many forced");
        uint256 nextIdx = IQueueForVerifier(inclusionQueueContract).nextVaultRequestToProcessIndex();
        require(forcedQueueIds[0] == nextIdx, "VRF: forced must start at pointer");
        for (uint256 k = 0; k < lenForced; k++) {
            require(forcedQueueIds[k] == nextIdx + k, "VRF: forced not contiguous");
        }

        // ... rest of the code
    }
}
```

These constraints mathematically dictate that `forcedQueueIds` must equal `[nextIdx, nextIdx + 1, nextIdx + 2, ..., nextIdx + lenForced - 1]`. There is no valid input that deviates from this sequence. Therefore, the array carries zero information beyond its length.

Additionally, the parameter is named `forcedQueueIds` suggesting request IDs. However, values are used as array indices when calling `poolQueue(forcedQueueIds[k])`.

Redundant `forcedQueueIds` array parameter wastes calldata gas and introduces unnecessary loop iterations for equality checks that enforce a fully deterministic, predictable sequence.

Assets:

- `./Verifier.sol` [zip file]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Replace the array with a simple count parameter:

```
function submitPoolUpdateBatch(
    // ... other params ...
    uint256 forcedCount // Replace: uint256[] calldata forcedQueueIds
) external onlySequencer whenNotPaused {
    // ... rest of the code

    if (forcedCount > 0) {
        require(forcedCount <= maxForcedPerBatch, "VRF: too many forced");
        uint256 nextIdx = IQueueForVerifier(inclusionQueueContract).nextPoolRe
questToProcessIndex();

        for (uint256 k = 0; k < forcedCount; k++) {
            uint256 queueIndex = nextIdx + k;
            // ... rest of the code
        }

        IQueueForVerifier(inclusionQueueContract).setPoolQueueIndex(forcedCoun
t);
    }

    // ... rest of the code
}
```

Resolution:

The issue was fixed in commit [322258e](#). The `forcedQueueIds` `uint256` array was replaced with a single `uint256` function parameter in both the `submitVaultUpdateBatch()` and `submitPoolUpdateBatch()` functions.

[F-2026-14512](#) - Floating Pragma - Info

Description:

In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8.20`) introduces uncertainty, as it allows contracts to be compiled with any version within a specified range. This can result in discrepancies between the compiler used in testing and the one used in deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The project currently uses floating pragma declarations (`^0.8.28`) in its Solidity contracts. This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

Assets:

- ./Verifier.sol [zip file]
- ./Pool.sol [zip file]
- ./InclusionQueue.sol [zip file]
- ./WithdrawalManager.sol [zip file]
- ./Governance.sol [zip file]
- ./Vault.sol [zip file]
- ./Pausable.sol [zip file]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

It is recommended to **lock the pragma version** to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler version, preventing unexpected changes in behavior due to compiler updates.

Resolution:

The issue was fixed in commit [322258e](#). The pragma was locked to version [0.8.30](#).

[F-2026-14513](#) - Custom Errors Can Be Used for Gas Efficiency - Info

Description:

Starting from Solidity version `0.8.4`, the language introduced a feature known as "custom errors". These custom errors provide a way for developers to define more descriptive and semantically meaningful error conditions without relying on string messages. Prior to this version, developers often used the `require` statement with string error messages to handle specific conditions or validations. However, every unique string used as a revert reason consumes gas, making transactions more expensive.

Custom errors, on the other hand, are identified by their name and the types of their parameters only, and they do not have the overhead of string storage. This means that, when using custom errors instead of `require` statements with string messages, the gas consumption can be significantly reduced, leading to more gas-efficient contracts.

Assets:

- `./Verifier.sol` [zip file]
- `./Pool.sol` [zip file]
- `./InclusionQueue.sol` [zip file]
- `./WithdrawalManager.sol` [zip file]
- `./Governance.sol` [zip file]
- `./Vault.sol` [zip file]
- `./Pausable.sol` [zip file]

Status:

Mitigated

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

It is recommended to use custom errors instead of revert strings to reduce increased Gas usage, especially during contract deployment. Custom

errors can be defined using the `error` keyword and can include dynamic information.

Current approach:

```
modifier onlyGovernance() {  
    require(msg.sender == governance, "VRF: not gov");  
    _;  
}
```

Usage of custom errors:

```
// Solidity version 0.8.26 or higher  
// custom error declaration  
error VerifierCallerIsNotGovernance();  
  
modifier onlyGovernance() {  
    require(msg.sender == governance, VerifierCallerIsNotGovernance());  
    _;  
}
```

Resolution:

The issue was mitigated in commit [322258e](#). The string error messages were replaced in almost all contracts, but `Pausable` still contains string error messages. Additionally, the contracts in scope use two different ways of validation and error handling: `require` statements and `if` statements.

[F-2026-14514](#) - Lack of Two-Step Ownership Transfer - Info

Description:

The `Pausable` contract currently utilizes the local copy of the `OwnableUpgradeable` library from OpenZeppelin for managing contract ownership. This library lacks a mechanism to ensure that ownership transfers are explicitly accepted by the new owner.

As a result, ownership can mistakenly be transferred to an address that cannot properly manage it, for example due to a typographical error or transfer to a non-contract address without the ability to perform administrative actions. Once such a transfer occurs, recovery may require governance intervention, upgrades, or redeployments, which increases operational risk.

Assets:

- `./Pausable.sol` [zip file]

Status:

Fixed

Classification

Impact Rate:

1/5

Likelihood Rate:

5/5

Exploitability:

Independent

Complexity:

Simple

Severity:

Info

Recommendations

Remediation:

It is recommended to replace the current `OwnableUpgradeable` implementation with OpenZeppelin's `Ownable2StepUpgradeable`. This approach introduces a two-step process in which the intended new owner must explicitly accept ownership. Such a mechanism reduces the risk of ownership being unintentionally transferred to an invalid or unusable address and provides stronger guarantees for safe contract administration.

Resolution:

The issue was fixed in commit `322258e`. The use of the `OwnableUpgradeable` library was replaced with `Ownable2StepUpgradeable`.

[F-2026-14518](#) - Violation of the Checks-Effects-Interactions (CEI) pattern - Info

Description:

The Checks-Effects-Interactions (CEI) pattern is a widely accepted best practice in smart contract development, designed to mitigate the risk of re-entrancy attacks. This pattern mandates that contracts should first perform checks, such as `require` statements, then execute effects, such as state changes, and finally carry out any interactions with external contracts. Failing to follow this pattern can expose the contract to re-entrancy attacks, where an attacker can repeatedly call a vulnerable function before its state is updated, potentially draining contract assets.

Some functions within the scope of this audit execute external calls prior to updating storage variables. Although these functions are protected with the `nonReentrant` modifier to prevent re-entrancy, they do not follow the CEI pattern. Deviating from CEI increases reliance on the modifier for security and reduces overall contract robustness.

Affected functions:

- Vault: `deposit()`,
- InclusionQueue: `forceWithdrawalFromPool()`, `forceWithdrawalFromVault()`,
- Pool: `depositToken()`,
- WithdrawalManager: `processOnChainPoolWithdrawal()`,
`processCrossChainPoolWithdrawal()`, `processVaultWithdrawal()`,
- Verifier: `submitPoolUpdateBatch()`, `submitVaultUpdateBatch()`,
`processOverdueQueueItem()`

Status:

Mitigated

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

It is recommended to refactor functions to follow the Checks-Effects-Interactions pattern, ensuring that all state changes occur before external calls.

Resolution:

The issue was mitigated in commit `322258e`. The CEI pattern was applied in almost all the mentioned functions, but the `deposit()` and `depositToken()` functions still violate the CEI pattern.

[F-2026-14519](#) - Unused Pausable Inheritance in WithdrawalManager

- Info

Description:

The `WithdrawalManager` contract imports and inherits from the `Pausable` contract but never utilizes any of its pause functionality. This represents dead code that increases deployment costs, contract bytecode size, and maintenance burden without providing any security benefit.

```
import {Pausable} from "../Pausable.sol";
```

The `WithdrawalManager` is designed as an internal contract that can only be called by the `Verifier` contract (enforced via `onlyVerifier` modifier). Since the Verifier already has pause functionality via `whenNotPaused` on its batch submission functions, pausing the entry point effectively pauses all downstream operations including `WithdrawalManager`.

Unused code inheritance increases deployment gas and creates confusion about whether this contract can be paused independently.

Assets:

- `./WithdrawalManager.sol` [zip file]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Remove the unused `Pausable` import and inheritance from `WithdrawalManager`.

Resolution:

The given finding is resolved by removing the `Pausable` import from the `WithdrawalManager` contract in commit `322258e`.

[F-2026-14520](#) - Shared Pool Nonce Invalidates Pending Signatures - Info

Description:

The `WithdrawalManager` contract uses a single shared nonce counter (`userPoolNonces`) for both on-chain and cross-chain pool withdrawals. When either withdrawal type is processed, the nonce is incremented, which invalidates any pending signatures of the other type that were created with the same nonce.

WithdrawalManager:

```
mapping(address => uint256) public userPoolNonces;
```

On-chain withdrawal checks and increments the shared nonce in

`processOnChainPoolWithdrawal()`:

```
require(nonce == userPoolNonces[user], "WM: bad nonce");
// ...
userPoolNonces[user] = nonce + 1;
```

Cross-chain withdrawal also checks and increments the same shared nonce in `processCrossChainPoolWithdrawal()`:

```
require(nonce == userPoolNonces[user], "WM: bad nonce");
// ...
userPoolNonces[user] = nonce + 1;
```

This design might create a race condition where:

1. User signs an on-chain withdrawal request with nonce N
2. User signs a cross-chain withdrawal request with nonce N (same nonce, different type)
3. Sequencer processes cross-chain withdrawal first → nonce becomes N+1
4. On-chain withdrawal signature is now invalid → reverts with "WM: bad nonce"

Users must re-sign withdrawal requests if another type is processed first and failed transactions consume gas.

Assets:

- `./WithdrawalManager.sol` [zip file]

Status:

Accepted

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:	Use separate nonce mappings for on-chain and cross-chain withdrawals. If the shared nonce is intentional (e.g., to enforce strict ordering), clearly document this behavior: • In NatSpec comments • In user-facing documentation
Resolution:	The BullBit team stated that the use of a shared nonce mapping for both on-chain and cross-chain withdrawals is an intentional design choice. Therefore, the given findings were accepted, and no further changes were applied.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/bullbit/Protocol-V1
Initial Commit	7766cc0501b540660fa8845f23592eb7620266f4
Final Commit	322258e4622b2430f1249853f2c0c013a038548f
Whitepaper	N/A
Requirements	https://docs.google.com/document/d/1LU-hAbZboob9CCYRedHqWi4wshr0okDLHA_fE6MIHwI/edit?usp=sharing
Technical Requirements	https://docs.google.com/document/d/1LU-hAbZboob9CCYRedHqWi4wshr0okDLHA_fE6MIHwI/edit?usp=sharing

Asset	Type
./Governance.sol [zip file]	Smart Contract
./InclusionQueue.sol [zip file]	Smart Contract
./Pausable.sol [zip file]	Smart Contract
./Pool.sol [zip file]	Smart Contract
./Vault.sol [zip file]	Smart Contract
./Verifier.sol [zip file]	Smart Contract
./WithdrawalManager.sol [zip file]	Smart Contract

Appendix 3. Additional Valuables

Verification of System Invariants

During the audit of BullBit / Protocol V1, Hacken followed its methodology by performing fuzz-testing on the project's main functions. [Foundry](#), a tool used for fuzz-testing, was employed to check how the protocol behaves under various inputs. Due to the complex and dynamic interactions within the protocol, unexpected edge cases might arise. Therefore, it was important to use fuzz-testing to ensure that several system invariants hold true in all situations.

Fuzz-testing allows the input of many random data points into the system, helping to identify issues that regular testing might miss. A specific Echidna fuzzing suite was prepared for this task, and throughout the assessment, 10 invariants were tested over 70000 runs. This thorough testing ensured that the system works correctly even with unexpected or unusual inputs.

Invariant	Test Result	Run Count
Queue indices should only ever increase, never exceed total	Passed	10k
User nonces should only ever be ≥ 0 (start at 0)	Passed	10k
Processed index should never exceed total requests	Passed	10k
Total deposits should be $\geq$ total withdrawals tracked	Passed	10k
Queue request count should be consistent	Passed	10k
Only the wallet owner can access assets via signed request	Passed	-
Overdue requests must be processable via escape hatch	Passed	-
Non-overdue requests should not be processed	Passed	-
Pool user internal balance sum will be lower/equal to the contract balance	Passed	10k
Vault user internal balance sum will be lower/equal to contract balance	Passed	10k

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.